

GENERATIVE NEURAL RETARGETING FOR HUMAN-TO-ROBOT DEXTEROUS MANIPULATION

Dechen Gao^{1,2,*†} Yue Yang^{1,3,*†} Ben Abbatematteo¹ Nathan Godwin¹
 Pengcheng Wang^{1,4†} Roger Boldu¹ Steven Man^{1,6†} Zhiyang Dou^{1,5†}
 Chuan Qin¹ Sho Nakagome¹ Eric Whitmire¹

¹Meta Reality Labs Research ²UC Davis ³UNC Chapel Hill
⁴UC Berkeley ⁵MIT ⁶CMU

*Equal contribution. †Work done at Meta.

Project Page | Correspondence: dcgao@ucdavis.edu, yygx@cs.unc.edu

ABSTRACT

Human demonstrations are a scalable data source for learning dexterous manipulation, but the embodiment gap prevents human motion from being executed directly on robots. Inverse kinematics (IK) retargets human motion to robots efficiently but ignores dynamics, often producing infeasible motions. Reinforcement learning (RL) and sampling-based model predictive control (MPC) are commonly employed to yield dynamically feasible motions, but both are sample-inefficient and sensitive to hyperparameters. RL suffers from costly and unstable training and tedious reward engineering; MPC avoids policy optimization, yet retargets each trajectory in isolation, and solving one does not make the next easier. Sampling cost grows rapidly with dataset size and task difficulty. We hypothesize that dynamically feasible trajectories concentrate near a low-dimensional manifold shared across demonstrations, so that retargeting can be reduced to sampling from that manifold, conditioned on human motion, rather than solving a fresh optimization problem for every demonstration. We propose **Generative Neural Retargeting** (GNR), which uses a flow matching model to sample feasible trajectories. GNR outperforms MPC with only 8.5% of the samples required by MPC, achieving a success rate of 56.20% compared to 27.20% for MPC. GNR can be used for scalable and efficient retargeting of large-scale, long-horizon, and millimeter precision human demonstrations: by applying GNR within a real-to-sim data engine, we produce a dexterous manipulation dataset with dense contact-force labels, spanning 223k demonstrations and 3.3k object geometries.

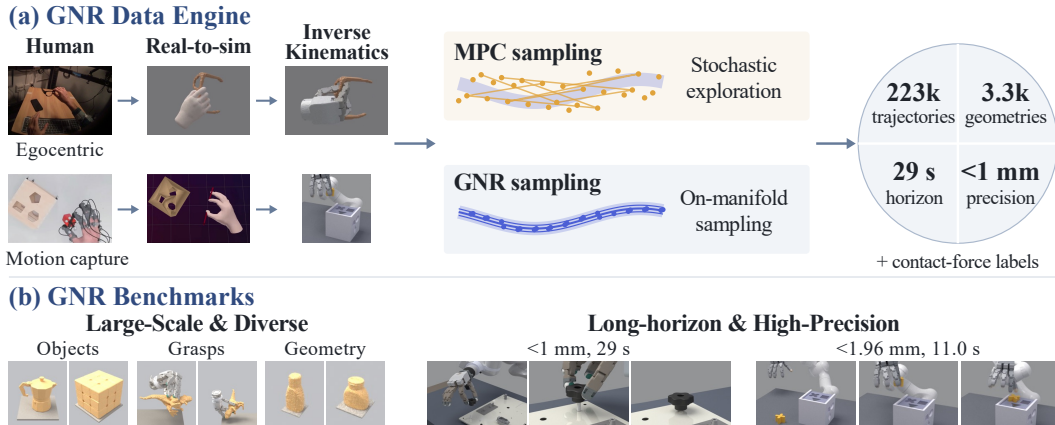


Figure 1: We propose **Generative Neural Retargeting** (GNR), a scalable and generalizable retargeting method that retargets large-scale, long-horizon and high-precision human demonstrations for five-fingered dexterous manipulation. We apply GNR with a real-to-sim data engine that reconstructs human motion from egocentric videos or a wearable exoskeleton with motion capture to scale up both diverse and difficult high-precision tasks.

1 INTRODUCTION

Progress in dexterous manipulation has been driven largely by the growing scale of demonstration data (O’Neill et al., 2024; Jiang et al., 2025; Hoque et al., 2026; Goel et al., 2023). Collecting such data directly on robots, e.g., through teleoperation, is prohibitively expensive: it demands extensive human labor and is constrained by hardware availability and embodiment-specific interfaces (Handa et al., 2020; Qin et al., 2023). A promising and scalable alternative is to record human hand manipulation from videos or wearable devices and transfer it to robots (Xu et al., 2025a; Banerjee et al., 2025; Zheng et al., 2026; Ma et al., 2026). However, a fundamental challenge in utilizing such data is that human demonstrations do not provide robot action labels, while the *embodiment gap* prevents human motions from being transferred directly to robots (Pan et al., 2025).

Prior work retargets human motions to robots through *kinematic* or *physics-based retargeting* (Pan et al., 2025; Mandi et al., 2025; Xu et al., 2025b;c; Allshire et al., 2025; Li et al., 2025; Qiu et al., 2026). Kinematic methods, e.g., fingertip IK, align human and robot poses geometrically. They are fast but ignore dynamics and contact, so the resulting trajectories are often dynamically infeasible (Handa et al., 2020; Qin et al., 2023). *Physics-based retargeting* instead optimizes for dynamically feasible motion through reinforcement learning (RL) (Qiu et al., 2026; Li et al., 2025; Allshire et al., 2025; Liao et al., 2026; Liu et al., 2025b; Adalibieke et al., 2026; Mandi et al., 2025; Tang et al., 2025) or sampling-based optimization methods such as model predictive control (MPC) (Pan et al., 2025; Williams et al., 2017; Xue et al., 2024). The retargeting objective is highly non-convex and discontinuous, which makes each trajectory expensive to solve. RL suffers from unstable and expensive training and tedious reward engineering (Liu et al., 2025b; Zhu et al., 2026; Mandi et al., 2025). Sampling-based MPC avoids costly policy optimization and directly optimizes the control sequence, but it can be sample-inefficient and sensitive to hyperparameters as every trajectory is solved with highly stochastic sampling and hand-designed heuristics (Pan et al., 2025; Paliwal et al., 2026; Mandi et al., 2025). The sample inefficiency and engineering burden create a scaling bottleneck for robot learning with human data, which motivates our research question: *how can we generate dynamically feasible robot trajectories from human demonstrations in a scalable and efficient way?*

A key observation is that conventional sampling-based retargeting repeatedly solves a non-convex optimization problem for every trajectory in isolation, and solving one does not make the next cheaper (Mandi et al., 2025; Liu et al., 2025b). However, solved trajectories can provide useful knowledge about feasible motion and contact when retargeting other demonstrations. Thus inspired, we propose **Generative Neural Retargeting** (GNR), a conditional flow matching model that learns from a small seed dataset generated by MPC to guide sampling toward dynamically feasible trajectories, conditioned on human motion. Scaling the training dataset can improve model performance (Yang et al., 2023), but generating the seed dataset can itself be expensive. GNR complements data scaling with *test-time scaling*: increasing the sampling budget improves retargeting success while reducing reliance on costly seed data. GNR generalizes to unseen motions, geometries, and objects, and even solves demonstrations on which MPC fails under the sampling budget used to generate GNR’s training data.

We apply GNR within a real-to-sim data engine to address the scaling bottleneck of retargeting along two axes, diversity and difficulty, for dexterous manipulation. Along the **diversity** axis, the goal is a dataset spanning many distinct geometries and motions, where total sampling cost grows linearly with the number of human trajectories, each of which can take hours to optimize (Pan et al., 2025; Mandi et al., 2025; Zhu et al., 2026). Therefore, state-of-the-art methods often demonstrate a few thousand trajectories derived from sources such as egocentric videos (Liu et al., 2025b; Adalibieke et al., 2026; Zhu et al., 2026; Paliwal et al., 2026), far short of the scale needed for large-scale, diverse dataset generation. Along the **difficulty** axis, the goal is solving long-horizon and high-precision dexterous manipulation. Errors compound over time, and millimeter-level precision leaves little tolerance for error, so the number of samples needed to discover a feasible trajectory grows rapidly (Ross et al., 2011; Yang et al., 2025). For example, a complex long-horizon industrial assembly task inspired by NIST (Lian et al., 2021) that requires 1 mm precision to screw a knob onto its seat (see Figure 21 for a demonstration) requires 221 CPU core-h to optimize a trajectory averaging 29 seconds. Along the difficulty axis, existing works primarily retarget human demonstrations on tasks with 10 to 100 millimeter precision requirements (Pan et al., 2025; Liu et al., 2025b; Mandi et al., 2025) due to both the lack of high-precision data sources and the massive sampling required for retargeting. We further discuss existing retargeting datasets in App. A.1.

As illustrated in Figure 1, the data engine takes human demonstrations from large-scale egocentric video and high-precision motion capture to scale along both axes. A real-to-sim pipeline transfers them into SuperDex (The Mochi Team, 2026) and applies domain randomization to object pose, physical properties, and geometry. Along the **diversity** axis, GNR outperforms state-of-the-art sampling-based methods such as MPC (Xue et al., 2024; Pinneri et al., 2021) with about 8.5% of their sampling budget on HOT3D after learning from a small seed dataset generated by MPC. On out-of-distribution (OOD) objects, motions, and geometries, it reaches 56.2% success versus around 30% for sampling-based methods, and solves 42.6% of demonstrations on which MPC fails. GNR reduces CPU cost per trajectory by $7.6\times$ relative to MPC. Along the **difficulty** axis, GNR solves long-horizon, millimeter-precision tasks open-loop or by supplying MPC with on-manifold initializations. CPU cost per trajectory falls by $14.9\times$ on ShapeFilter and $3.0\times$ on NIST. Test-time scaling complements data scaling: with only 50 training demonstrations on ShapeFilter, GNR success rises from 11.5% to 99.5% as the sample budget grows to 256. Our contributions are:

Retargeting as generative modeling with test-time scaling. We recast physics-based retargeting from per-trajectory sampling-based optimization to generative modeling, GNR, which allows for efficient and generalizable robot action sampling conditioned on human motion. GNR demonstrates test-time scaling and generalizes to unseen geometries and motions, making it a scalable and cost-efficient method for retargeting large-scale, long-horizon, and high-precision human demonstrations to feasible robot trajectories.

A scalable and generalizable data engine. We develop a real-to-sim pipeline that extracts and retargets human demonstrations from egocentric videos and motion capture at scale. From HOT3D, it produces 223k paired human and feasible robot trajectories across 3.3k object geometries with dense contact-force labels. We also produce 3.4k retargeted trajectories for two long-horizon tasks requiring millimeter precision.

2 RELATED WORK

Dexterous retargeting converts human demonstrations into robot trajectories (Pan et al., 2025; Mandi et al., 2025; Liu et al., 2025b; Zhu et al., 2026; Paliwal et al., 2026; Handa et al., 2020; Qin et al., 2023; Bahety et al., 2024; 2025; Abbatematteo et al., 2020; Abdul-Rashid et al., 2022). Kinematic retargeting aligns human and robot keypoints (Dariush et al., 2008a; Rohling & Hollerbach, 1993; Antotsiou et al., 2018; Dariush et al., 2008b; Yin et al., 2025; Wu et al., 2026a), but does not enforce dynamic feasibility. Physics-based retargeting instead aims at generating feasible trajectories through reinforcement learning (RL) (Liu et al., 2025b; Zhu et al., 2026; Mandi et al., 2025) or sampling-based optimization (Pan et al., 2025; Paliwal et al., 2026; Xue et al., 2024). RL suffers from tedious reward engineering and unstable, costly policy optimization. Sampling-based methods such as MPC avoid costly policy training but optimize each trajectory in isolation and fail to utilize knowledge in solved trajectories. Generative predictive control (Kurtz & Burdick, 2025) distills sampling-based MPC into a separate policy for each task. GNR instead conditions on the human demonstration, so a single model amortizes retargeting across demonstrations and generalizes to unseen motions and objects. Additional related work is discussed in App. A.

Real-to-sim methods reconstruct simulation environments from real data (Torne et al., 2024; Jiang et al., 2022; Dai et al., 2024; Chen et al., 2024; Fang et al., 2025), and have been utilized for robot data generation from human demonstrations (Zhang et al., 2026; Qin et al., 2022). These datasets typically omit contact-force labels, with DexCanvas (Xu et al., 2025c) as an exception, but its labels describe interactions with an actuated MANO human-hand model rather than a robot hand. Our data engine labels contact forces on the robot hand and also uses GNR for large-scale and high-precision retargeting, scaling to 223k physically feasible trajectories across 3.3k geometries.

3 GENERATIVE NEURAL RETARGETING

3.1 RETARGETING AS A GENERATIVE PROBLEM

Physics-based retargeting problem. Retargeting takes a human demonstration, represented either as human hand motion or as a robot kinematic reference obtained by IK, and asks for a dynamically feasible control sequence $\mathbf{u}_{0:T-1}$ over T timesteps that reproduces the demonstrated behavior under

physics. We denote the reference motion as $\bar{\mathbf{x}}_{0:T} = (\bar{\mathbf{x}}_0, \dots, \bar{\mathbf{x}}_T)$, where each $\bar{\mathbf{x}}_t$ contains the robot and object states at timestep $0 \leq t \leq T$. We follow the physics-based retargeting formulation that casts retargeting as a constrained optimization problem (Pan et al., 2025), $\min_{\mathbf{u}_{0:T-1}} J(\mathbf{u}_{0:T-1})$, subject to $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$ for $0 \leq t < T$, $\mathbf{x}_{0:T} \in \mathcal{X}$, and $\mathbf{u}_{0:T-1} \in \mathcal{U}$, where $\mathbf{x}_t$ is the simulated robot and object state, f is the dynamics, $\mathcal{X}$ and $\mathcal{U}$ are the sets of admissible state and control sequences, and J is the retargeting cost (e.g., object tracking errors and force regularization).

The scaling bottleneck of sampling-based optimization. The retargeting problem is highly non-convex and discontinuous due to the contact-rich nature of dexterous manipulation (Pan et al., 2025). Sampling-based optimizers solve it by numerous stochastic rollouts (Pan et al., 2025; Mandi et al., 2025; Xue et al., 2024) but exploring with task-agnostic noise makes it sample-inefficient to discover feasible trajectories. Thus, several methods hand-design heuristics to mitigate the issue through, e.g., contact guidance, annealed noise schedules (Pan et al., 2025), and virtual spring-damper constraints (Mandi et al., 2025). Sample inefficiency and heuristics limit scalability. Furthermore, each trajectory is optimized independently, which fails to transfer knowledge across demonstrations.

Generative modeling with test-time scaling. We posit that feasible trajectories concentrate on a low-dimensional manifold that can be learned by a model that samples from it. Given a set $\mathcal{D}$ of human demonstrations, GNR learns from a seed dataset of N demonstrations retargeted by sampling-based methods such as DIAL-MPC (Xue et al., 2024), then retargets the remaining $|\mathcal{D}| - N$ demonstrations. Increasing N can improve model performance but requires costly seed generation. Test-time scaling complements data scaling by improving success through sampling, reducing reliance on seed datasets. Notably, GNR can outperform the seed generator and solve demonstrations on which it fails, expanding dataset coverage while reducing generation cost.

Flow-matching (Lipman et al., 2022). We model GNR as a conditional flow matching generator $G_\theta(\cdot | \mathbf{c})$ of residual corrections $\Delta = \mathbf{u}_{0:T-1} - \bar{\mathbf{u}}_{0:T-1}$ to the IK reference controls $\bar{\mathbf{u}}_{0:T-1}$, conditioned on $\mathbf{c} = (\bar{\mathbf{x}}_{0:T}, \mathbf{o})$, where $\mathbf{o}$ encodes object geometry or pose, depending on the task. For training pairs $(\Delta, \mathbf{c})$ from the optimizer-generated data, we independently sample standard Gaussian noise $\mathbf{z}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and flow time $\tau \sim \mathcal{U}[0, 1]$, set $\mathbf{z}_\tau = (1 - \tau)\mathbf{z}_0 + \tau\Delta$, and train the velocity field $\mathbf{v}_\theta$ with the flow matching loss $\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{(\Delta, \mathbf{c}), \mathbf{z}_0, \tau} [\|\mathbf{v}_\theta(\mathbf{z}_\tau, \tau, \mathbf{c}) - (\Delta - \mathbf{z}_0)\|_2^2]$. At inference, we draw Gaussian noise $\mathbf{z}_0$ and integrate $d\hat{\mathbf{z}}_\tau/d\tau = \mathbf{v}_\theta(\hat{\mathbf{z}}_\tau, \tau, \mathbf{c})$ from $\hat{\mathbf{z}}_0 = \mathbf{z}_0$ to $\tau = 1$, yielding the residual $\hat{\Delta} = G_\theta(\mathbf{z}_0 | \mathbf{c}) = \hat{\mathbf{z}}_1$ and retargeted trajectory $\mathbf{u}_{0:T-1} = \bar{\mathbf{u}}_{0:T-1} + \hat{\Delta}$. App. E.4 explains the design choice of predicting residuals, which outperforms predicting absolute controls.

3.2 GNR DEPLOYMENT

Open-Loop GNR Retargeting. GNR enables test-time scaling through its generative formulation: for a condition $\mathbf{c}$, we can draw S latents $\{\mathbf{z}_0^{(k)}\}_{k=1}^S$ and decode S candidate trajectories $\{\mathbf{u}_{0:T-1}^{(k)}\}_{k=1}^S$. Each candidate is replayed in simulation and checked against the task success criterion (see Table 4). Retargeting succeeds if any of the S candidates passes, which we report as $\text{pass}@S$.

GNR for MPC Initialization. Closed-loop feedback remains valuable for long-horizon and high-precision tasks, where small errors compound over time and lead to catastrophic failure. GNR integrates naturally with MPC by supplying a high-quality on-manifold initialization, $\mathbf{u}_{0:T-1} = \bar{\mathbf{u}}_{0:T-1} + \hat{\Delta}$, which MPC then refines.

4 A HUMAN-TO-ROBOT DATA ENGINE

GNR accelerates physics-based retargeting and drives a human-to-robot data engine (Figure 2) which produces feasible robot trajectories with simulation-derived contact-force annotations across both difficulty and diversity axes. We describe the retargeting pipeline in Section 4.1 and the discovery and scaling phases for expanding dataset diversity in Section 4.2.

4.1 HUMAN-TO-ROBOT RETARGETING

The data engine takes per-frame human hand motion, object poses, and an object surface mesh $\mathcal{M}$ as input. We segment multi-skill recordings using hand-object contact estimated from finger-to-surface distances, while using single-skill recordings directly. Object meshes are repaired to obtain

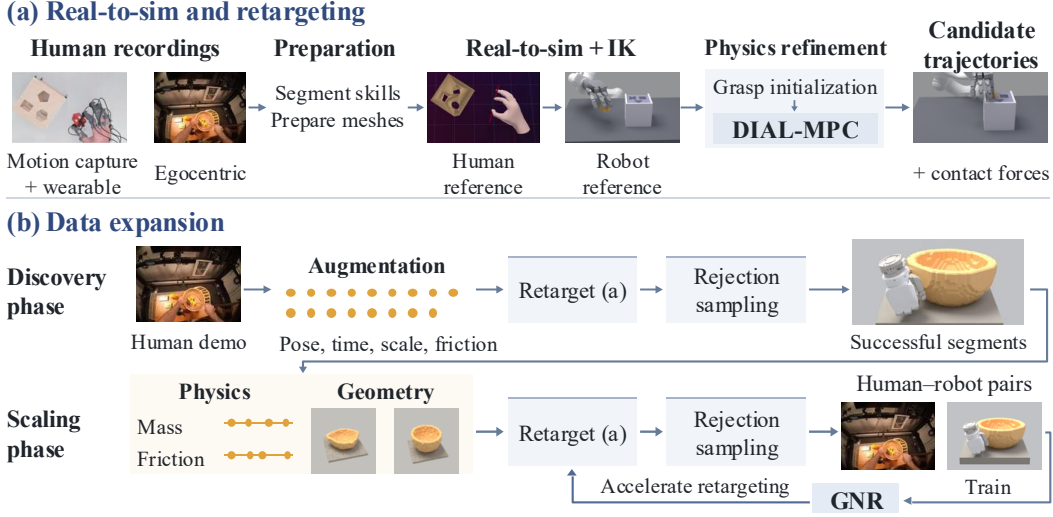


Figure 2: **The human-to-robot data engine.** (a) A pipeline retargets motion-capture recordings for the difficulty axis and egocentric demonstrations for the diversity axis into candidate robot trajectories with simulation-derived contact-force labels. (b) The discovery and scaling phases repeatedly apply (a) under varied conditions and filter candidates using the same success criterion Γ (Section 4.2), first identifying retargetable segments and then expanding their physical and geometric coverage. The resulting pairs train GNR to accelerate subsequent retargeting.

watertight surfaces, and their signed distance fields Φ are precomputed for penetration checks. We transfer the captured motion to a simulated human hand, then apply keypoint-based IK to retarget it to the robot. Together with the recorded object poses, the resulting robot configurations form the kinematic reference trajectory $\bar{x}_{0:T}$. For segments starting with an object already grasped, we re-establish the grasp by opening the robot hand, closing its fingers around the temporarily fixed object, and then releasing the object to move freely in simulation. To construct the initial training pairs for GNR, we use DIAL-MPC (Xue et al., 2024) to optimize robot controls that track the reference object motion, recording simulation-derived contact forces during replay. Once trained, GNR accelerates subsequent retargeting in open loop or by initializing MPC for refinement. Implementation details are provided in App. C. For the difficulty axis, individually recorded motion-capture demonstrations are converted into paired training data through this pipeline without skill segmentation. For the diversity axis, egocentric demonstrations undergo the additional stages below.

4.2 TWO-PHASE DATA EXPANSION

To improve data diversity, we apply a two-phase data expansion pipeline to egocentric demonstrations (Figure 2b). The discovery phase identifies retargetable segments, while the scaling phase expands them across object geometry and physical properties.

Discovery Phase. To avoid costly expansion of unsuccessful segments, we first search for feasible retargeting under a small set of candidate conditions. Reconstruction errors and the embodiment gap can hinder retargeting under the recorded conditions. We therefore search for feasible solutions by optimizing each segment under the nominal condition and sixteen single-axis perturbations of object position, orientation, start time, scale, or surface friction. However, low object-tracking error alone does not ensure valid physical interaction. Grasp initialization reduces initial penetration but does not prevent it during subsequent motion, and optimized controls may still produce excessive contact forces. For demonstrations with little object motion, low tracking error can also occur without a sustained grasp. We therefore retain only simulated trajectories $\xi = (\mathbf{x}_{0:T}, \mathbf{u}_{0:T-1})$ with $\Gamma(\xi) = 1$, where $\Gamma(\xi) = \gamma_{\text{track}}(\xi) \wedge \gamma_{\text{pen}}(\xi) \wedge \gamma_{\text{grasp}}(\xi) \wedge \gamma_{\text{force}}(\xi)$. Here, γ_{track} bounds object position and orientation errors relative to the reference; γ_{pen} bounds finger-capsule penetration using Φ ; γ_{grasp} requires a minimum sustained squeeze force; and γ_{force} limits per-finger contact force. Thresholds are provided in Table 4. Accepted trajectories form the discovery dataset $\mathcal{D}_1$; only segments with at

least one accepted trajectory proceed to the scaling phase. We also use Γ to evaluate GNR trajectories on HOT3D.

Scaling Phase. We expand successfully retargeted segments by varying object physics and geometry. For physical variation, we resample surface friction and compute object mass from mesh volume and a density sampled from an object-specific range estimated by a large language model. Geometric variation includes mesh regeneration from the same image with different random seeds, anisotropic deformation, completion from partially masked images, regeneration after changing the object’s orientation about its vertical axis, and single-axis stretching. We use SAM3D for image-based mesh generation and apply deformation and stretching directly to the meshes. For each variant, we optimize control sequences under the modified conditions using multiple optimizer seeds and retain trajectories satisfying the same criterion Γ used in discovery. The additional accepted trajectories form $\mathcal{D}_2$, yielding the MPC-generated robot dataset $\mathcal{D}_{\text{robot}} = \mathcal{D}_1 \cup \mathcal{D}_2$.

5 EXPERIMENTS

We evaluate GNR along the diversity and difficulty axes. We first describe the experimental settings (Section 5.1), then evaluate GNR’s performance and efficiency on large-scale, diverse egocentric demonstrations (Section 5.2) and long-horizon, high-precision demonstrations (Section 5.3).

5.1 EXPERIMENTAL SETUP

Simulator & Embodiment. We use SuperDex Physics (The Mochi Team, 2026) as the simulator, which supports non-convex collision geometry and spatially distributed 3-D contact forces, enabling dense contact-force annotations. The target embodiment is a five-fingered Tesollo DG-5F hand for HOT3D, and a DG-5F hand mounted on a Franka FR3 arm for ShapeFilter and NIST.

GNR Training and Sampling. We implement GNR as a conditional flow matching model, which predicts the residual to correct the IK reference. The backbone is a temporal U-Net (Ronneberger et al., 2015) with feature-wise linear modulation (FiLM) (Perez et al., 2018) to inject the IK reference and geometry conditioning. We use a Point Transformer V3 (Wu et al., 2024) encoder over 1,024 points as the geometry condition. We sample from GNR with an Euler ordinary differential equation (ODE) solver (Lipman et al., 2022); model-specific settings are in App. B. We write S for the number of samples drawn per demonstration. Multiple samples can be drawn concurrently in a batch.

MPC Sampling for Seed Dataset Generation. We denote samples per MPC iteration by n_s and the maximum number of MPC iterations in each optimization by M . Following the retargeting workflow in Section 4.1, we use DIAL-MPC (Xue et al., 2024) as both a retargeting baseline and the seed dataset generator for all tasks. We use 300 samples for 10 MPC iterations on HOT3D and 500 samples for 6 iterations on ShapeFilter. For NIST, the three stages use 86, 86, and 172 samples, respectively, for 10 iterations per stage.

Baselines. On HOT3D, we compare against the sampling-based methods MPPI (Williams et al., 2017), DIAL-MPC (Xue et al., 2024), iCEM (Pinneri et al., 2021), and CMA-ES (Hansen & Ostermeier, 2001), which use the cost function J in Section 3.1. Baseline results for HOT3D are discussed in Section 5.2; additional results for ShapeFilter are in App. E.3. We also compare GNR against baselines like SPIDER (Pan et al., 2025) on their public benchmarks (see App. H.2).

Metrics. The primary metric is success rate (SR), measured using task-specific criteria (Table 4, App. B). We additionally report object position error E_{pos} , object orientation error E_{rot} , and finger-joint angle error E_{finger} relative to the reference trajectory (Mandi et al., 2025; Liu et al., 2025b; Zhu et al., 2026). We compare CPU cost per trajectory in core-hours (core-h) or core-seconds (core-s), including inference and simulation.

5.2 SCALING TO LARGE-SCALE AND DIVERSE RETARGETING

This section demonstrates GNR’s sample efficiency and generalization advantages that enable scalable and efficient retargeting of large, diverse collections of human demonstrations.

Collecting demonstrations from egocentric data. Egocentric data has emerged as a cost-efficient source to expand data diversity and quantity for robot learning (Zheng et al., 2026; Ma et al., 2026). We use human demonstrations from the egocentric HOT3D dataset (Banerjee et al., 2025), extracting 780 source motions covering 33 objects with an average duration of approximately 2 s. The data engine in Section 4 transfers these motions into simulation and constructs 360k augmented human demonstrations by varying object pose, geometry, physical properties, and motion timing.

Retargeting coarse motion references. However, reconstructed human motions from videos are often infeasible due to reconstruction errors. IK exacerbates infeasibility due to the embodiment gap. We therefore use physics-based retargeting. We employ DIAL-MPC (Xue et al., 2024), which succeeds on 95k of 360k demonstrations (26.43%) and requires 390k CPU core-h, equivalent to 31.7 days on 512 CPU cores. We refer to the successful and unsuccessful demonstrations as the MPC-success (95k) and MPC-failure (264k) datasets, respectively.

GNR training and evaluation. The costly MPC generation motivates the key questions: *does GNR outperform MPC while reducing sampling, and does GNR generalize to OOD or MPC-failure demonstrations?* To examine this, we randomly sample a small training dataset for GNR using 28k demonstrations (about 30% of the 95k MPC-success) covering 23 of 33 objects. Because running sampling-based baselines is prohibitively expensive, we randomly sample 1,000 test demonstrations from those with an unseen object (203), source grasp (338), or geometry variant (459); 272 are MPC successes and 728 are MPC failures.

We compare GNR’s SR and efficiency with state-of-the-art retargeting methods in Table 1. Fingertip IK directly matches the fingertip reference and thus has the lowest E_{finger} , but achieves only 18.3% success. MPPI, DIAL-MPC, iCEM, and CMA-ES optimize costs that include object tracking errors and generally achieve lower E_{pos} and E_{rot} . However, optimization can get trapped in local minima with low tracking errors that still violate grasp or contact constraints, so lower tracking errors alone do not ensure success. These methods achieve only 27.2–34.7% success with 3,000 samples, whereas open-loop GNR reaches 52.2% and 56.2% with 128 and 256 samples, respectively. Compared with its seed generator DIAL-MPC, GNR reduces CPU cost per trajectory by 7.6 $\times$ while increasing SRs from 27.2% to 56.2%.

Table 1: Performance comparison on HOT3D, split into MPC-success and MPC-failure datasets.

Method	S	Overall			MPC-success			MPC-failure			
		core-s	SR (%)	SR (%)	E_{pos} (mm)	E_{rot} ($^{\circ}$)	E_{finger} ($^{\circ}$)	SR (%)	E_{pos} (mm)	E_{rot} ($^{\circ}$)	E_{finger} ($^{\circ}$)
IK	1	39.0	18.30	50.00	34.92	16.38	0.10	6.46	54.52	34.59	0.13
DIAL-MPC	3,000	2689.3	27.20	100.00	25.33	10.98	1.15	0.00	36.29	21.73	1.49
MPPI	3,000	1849.6	27.70	71.32	28.04	11.96	1.22	11.40	36.92	20.59	2.07
iCEM	3,000	3862.1	34.70	78.68	21.96	7.07	1.74	18.27	26.04	10.92	2.14
CMA-ES	3,000	2912.7	32.00	72.79	23.21	8.55	1.02	16.76	32.89	14.55	1.08
GNR	1	48.9	18.10	48.16	34.96	16.26	1.22	6.87	54.48	33.07	1.24
GNR	64	90.9	49.60	88.97	27.82	13.74	1.05	34.89	37.10	18.29	0.97
GNR	128	178.6	<u>52.20</u>	<u>90.07</u>	28.04	13.74	1.05	<u>38.05</u>	36.18	17.64	0.99
GNR	256	354.1	56.20	92.65	28.04	13.99	1.05	42.58	36.18	17.58	0.98

* E_{pos} , E_{rot} , E_{finger} : object position, orientation, and finger-joint angle errors (medians); CPU core-s: mean per demonstration.

GNR’s generalization. Figure 3 reports GNR’s performance on three OOD subsets. Test-time scaling improves SR across all three: on MPC-success demonstrations, the mean SR across subsets rises from 46.2% at $S=1$ to 93.2% at $S=256$. With $S=256$, GNR also solves 31.9–58.4% of demonstrations that MPC fails to solve with 3,000 samples, expanding coverage beyond the seed optimizer with fewer samples.

We apply GNR trained on 28k seed trajectories to the 360k augmented demonstrations with up to $S=256$ samples, only 8.5% of MPC’s sampling budget. On the 264k MPC-

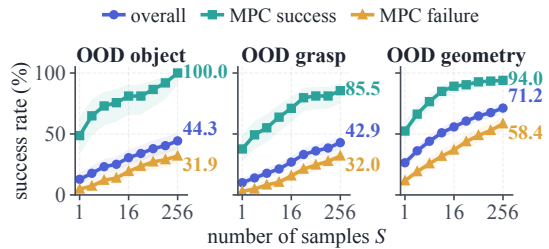


Figure 3: GNR performance on HOT3D across three OOD subsets. For each subset, we report GNR’s SRs on MPC-success, MPC-failure, and all demonstrations.

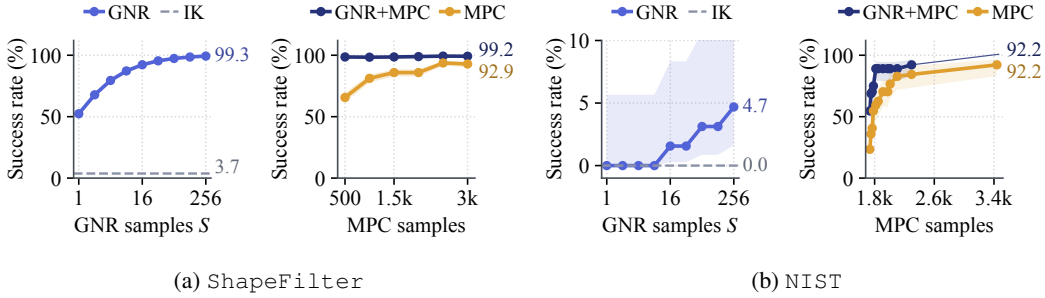


Figure 4: GNR’s performance on high-precision and long-horizon tasks. On ShapeFilter, open-loop GNR improves with the number of samples S , and GNR warm starts reduce MPC sampling requirements. On NIST, open-loop sampling alone is insufficient, while using GNR to initialize MPC (GNR+MPC) outperforms MPC from scratch at matched sampling budget.

failure demonstrations, GNR solves 127,136 (48.01%). Together with the 95k MPC-success trajectories, this expands the dataset to 223k feasible trajectories across 3.3k geometries. On the MPC-success demonstrations, GNR achieves 89.30% SR. These results support using GNR as a primary generator for large-scale human-to-robot datasets. See App. D.4 for details.

GNR’s efficiency. We estimate the cost of a practical deployment in which DIAL-MPC generates a small seed dataset and GNR retargets the remaining demonstrations. DIAL-MPC alone requires 390k CPU core-h to retarget 360k demonstrations, yielding 95k successful trajectories. Obtaining 28k seed demonstrations requires about 108k MPC attempts and 116.1k core-h. We run GNR on all the remaining 332k demonstrations at $S=256$, which costs only 32.6k core-h. GNR solves 127,136 MPC-failure demonstrations and achieves 88.91% SR on the 67k MPC-success demonstrations outside the seed set. Therefore, GNR generates 215k trajectories at 148.7k CPU core-h, a $2.62\times$ reduction relative to DIAL-MPC.

5.3 SOLVING HIGH-PRECISION AND LONG-HORIZON RETARGETING

This section examines the scaling challenges along the difficulty axis where we use high-precision motion capture to collect human demonstrations for more complex tasks, and employ GNR to improve dataset generation efficiency. Furthermore, we apply GNR-generated data for policy learning using behavioral cloning, and the results are discussed in App. I.1.

Scaling challenges for high-precision and long-horizon retargeting. Existing dexterous retargeting methods primarily focus on shorter tasks that tolerate centimeter-level errors (Pan et al., 2025; Liu et al., 2025b; Xu et al., 2025b; Zhu et al., 2025) (see Table 3 for details), whereas precise manipulation often demands millimeter precision over long horizons. Scaling along this axis presents two challenges. *First*, human motion recovered from visual observations, as in HOT3D, lacks the precision needed for effective optimization. *Second*, RL and MPC require significantly more samples to optimize longer trajectories under tighter precision constraints, motivating higher-quality human motion references and more sample-efficient retargeting.

Accordingly, we collect human demonstrations using 15 OptiTrack cameras and an instrumented wearable exoskeleton to improve human motion precision (Furtado et al., 2019). We consider two tasks: 1) ShapeFilter inserts a cross-shaped part into a matching hole with a tolerance of 1.96 mm; its dataset contains 3,037 demonstrations augmented from 605 human demonstrations. 2) NIST, an industrial assembly task (Lian et al., 2021), involves grasping, transporting, aligning, and screwing a knob onto a threaded post; success requires horizontal error below 1 mm, screwing down by at least 8 mm, and axis error below 0.003 rad. Its dataset contains 358 demonstrations. Our real-to-sim pipeline reconstructs both tasks in SuperDex, where DIAL-MPC optimizes each reference trajectory for feasibility, providing the seed datasets and the baseline. Dataset and optimization details are in App. E and App. F; demonstration examples are in App. J.1 and App. J.2.

GNR training and evaluation. We use 2,448/589 training/test demonstrations for ShapeFilter and 294/64 for NIST. Following Section 3.2, we evaluate GNR open-loop and as an on-manifold initialization for MPC (denoted as GNR+MPC). On ShapeFilter, IK achieves only 3.7% SR,

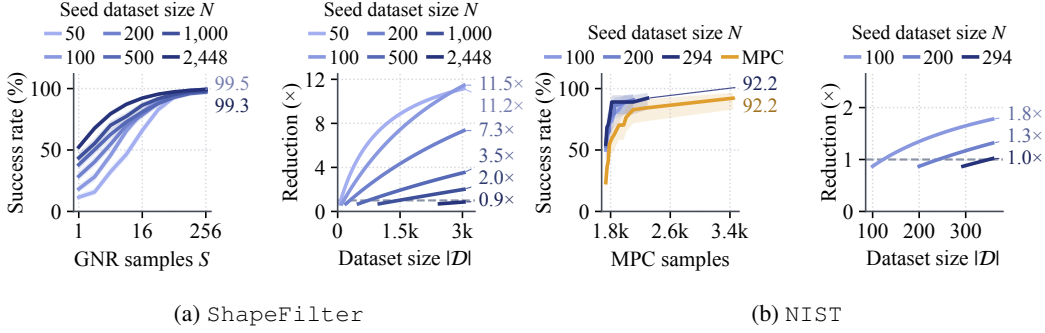


Figure 5: GNR’s efficiency in data generation. For each task, left: test-time scaling using GNR trained with different seed dataset sizes N . Right: CPU cost reduction as dataset size $|D|$ grows. Costs include MPC seed generation and GNR generation.

while open-loop success rises from 52.4% at $S=1$ to 97.4% at $S=64$ and 99.3% at $S=256$. At $S=64$, GNR exceeds MPC’s 92.9% SR with 3,000 samples ($M=6$, $n_s=500$), using only 1.6% of its CPU cost. Combined with MPC, GNR achieves 98.4% to 99.3% SR across $M=1$ to $M=6$ iterations and consistently outperforms vanilla MPC initialized from IK at each M , showing the benefit of on-manifold initialization (Figure 4a). On NIST, open-loop GNR reaches only 4.7% SR at $S=256$, compared with 0% for IK (Figure 4b), motivating closed-loop feedback for timely corrections over long horizons. With GNR initialization, MPC reaches 92.2% SR using 2,296 samples instead of the 3,440 used by vanilla MPC, a 33.3% reduction (Figure 4b).

Data and test-time scaling of GNR. Larger seed datasets can improve GNR’s performance but require more costly MPC generation of the seed dataset. We ablate the seed dataset size N in Figures 5a and 5b. On ShapeFilter, open-loop GNR success at $S=1$ increases from 11.5% with 50 training demonstrations to 52.4% with 2,448. Even with only 50 demonstrations, test-time scaling raises success to 99.5% at $S=256$. On the contact-rich NIST task, we use GNR to provide on-manifold initializations for MPC. Increasing the number of MPC samples raises success from 54.7% to 92.2% with $N=294$ training demonstrations and from 50.0% to 90.6% with $N=100$ demonstrations. GNR models trained with $N=294$ and $N=100$ use 33.3% and 38.8% fewer MPC samples, respectively, than IK-initialized MPC, which achieves 92.2% success. These results show that test-time scaling complements data scaling on both tasks, reducing reliance on costly MPC-generated seed data. See App. G for details.

GNR’s efficiency. Table 2 reports per-trajectory success and CPU cost: GNR reduces cost by 14.9 $\times$ on ShapeFilter and 3.0 $\times$ on NIST when achieving similar SRs. We then compute the total cost of generating each dataset, including MPC generation of N successful seeds and retargeting the remaining $|D|-N$ demonstrations with GNR or GNR+MPC. For 3,037 ShapeFilter demonstrations, $N=50$, $S=77$ gives a 94.8% dataset success rate at 765.6 CPU core-h, versus 93.7% at 8,541.0 core-h for MPC, an 11.2 $\times$ compute reduction (Figure 5a). For 358 NIST demonstrations, $N=100$ with a total MPC sample budget of 1,976 gives a 92.1% dataset success rate at 44,516.7 core-h, versus 92.2% at 79,135.6 core-h for MPC, a 1.8 $\times$ reduction (Figure 5b).

Table 2: Comparing efficiency and SR using MPC and GNR.

Method	Samples	SR (%) $\uparrow$	core-s $\downarrow$
ShapeFilter			
MPC	2,500	93.7	10,124
GNR	77	94.7	678
NIST			
MPC	3,440	92.2	795,777
GNR+MPC	1,976	89.1	266,199

6 CONCLUSION

We propose Generative Neural Retargeting (GNR), which learns from dynamically feasible trajectories and amortizes physics-based retargeting across demonstrations. Increasing test-time sampling boosts GNR’s success, complementing data scaling when feasible training trajectories are costly. GNR generalizes across motions and geometries, achieves higher success than MPC with far fewer samples, and retargets demonstrations on which MPC fails. We apply GNR within a real-to-sim data

engine that scales retargeting along two axes: diversity, producing a retargeting dataset with 223k robot trajectories spanning 3.3k geometries; and difficulty, producing 3.4k robot trajectories for two challenging millimeter-precision, long-horizon tasks. We demonstrate that behavioral cloning policies trained on GNR-generated data can succeed in simulation. Future work may study sim-to-real transfer of the learned policy.

REFERENCES

- Ben Abbatematteo, Stefanie Tellex, and George Konidaris. Learning to generalize kinematic models to novel objects. In Leslie Pack Kaelbling, Danica Kragic, and Komei Sugiura (eds.), *Proceedings of the Conference on Robot Learning*, Proceedings of Machine Learning Research, pp. 1289–1299. PMLR, 30 Oct–01 Nov 2020. URL <https://proceedings.mlr.press/v100/abbatematteo20a.html>.
- Hameed Abdul-Rashid, Miles Freeman, Ben Abbatematteo, George Konidaris, and Daniel Ritchie. Learning to infer kinematic hierarchies for novel object instances. In *2022 International Conference on Robotics and Automation (ICRA)*, pp. 8461–8467. IEEE, 2022.
- Jianibieke Adalibieke, Qianwei Han, Xueyi Liu, Yuzhe Qin, and Li Yi. Adadextrack: Dynamic modulation for adaptive and generalizable dexterous manipulation tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 28021–28031, June 2026.
- Arthur Allshire, Hongsuk Choi, Junyi Zhang, David McAllister, Anthony Zhang, Chung Min Kim, Trevor Darrell, Pieter Abbeel, Jitendra Malik, and Angjoo Kanazawa. Visual imitation enables contextual humanoid control. *arXiv preprint arXiv:2505.03729*, 2025.
- Dafni Antotsiou, Guillermo Garcia-Hernando, and Tae-Kyun Kim. Task-oriented hand motion retargeting for dexterous manipulation imitation, 2018. URL <https://arxiv.org/abs/1810.01845>.
- Arpit Bahety, Priyanka Mandikal, Ben Abbatematteo, and Roberto Martín-Martín. Screwmimic: Bimanual imitation from human videos with screw space projection. *arXiv preprint arXiv:2405.03666*, 2024.
- Arpit Bahety, Arnav Balaji, Ben Abbatematteo, and Roberto Martín-Martín. Safemimic: Towards safe and autonomous human-to-robot imitation for mobile manipulation. *arXiv preprint arXiv:2506.15847*, 2025.
- Prithviraj Banerjee, Sindi Shkodrani, Pierre Moulon, Shreyas Hampali, Shangchen Han, Fan Zhang, Linguang Zhang, Jade Fountain, Edward Miller, Selen Basol, et al. Hot3d: Hand and object tracking in 3d from egocentric multi-view videos. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7061–7071. IEEE, 2025.
- Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, pp. 306–324. Springer, 2024.
- Boyu Chen, Yi Chen, Lu Qiu, Jerry Bai, Yuying Ge, and Yixiao Ge. Unit: Toward a unified physical language for human-to-humanoid policy learning and world modeling. *arXiv preprint arXiv:2604.19734*, 2026a.
- Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, et al. SAM 3D: 3Dfy anything in images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7220–7232, 2026b.
- Zoey Chen, Aaron Walsman, Marius Memmel, Kaichun Mo, Alex Fang, Karthikeya Vemuri, Alan Wu, Dieter Fox, and Abhishek Gupta. Urdformer: A pipeline for constructing articulated simulation environments from real-world images. *arXiv preprint arXiv:2405.11656*, 2024.

- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Tianyuan Dai, Josiah Wong, Yunfan Jiang, Chen Wang, Cem Gokmen, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. Automated creation of digital cousins for robust policy learning. *arXiv preprint arXiv:2410.07408*, 2024.
- Behzad Dariush, Michael Gienger, Arjun Arumbakkam, Christian Goerick, Youding Zhu, and Kikuo Fujimura. Online and markerless motion retargeting with kinematic constraints. In *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 191–198. IEEE, 2008a.
- Behzad Dariush, Michael Gienger, Bing Jian, Christian Goerick, and Kikuo Fujimura. Whole body humanoid control from human motion descriptors. In *2008 IEEE International Conference on Robotics and Automation*, pp. 2677–2684. IEEE, 2008b.
- Yu Fang, Yue Yang, Xinghao Zhu, Kaiyuan Zheng, Gedas Bertasius, Daniel Szafir, and Mingyu Ding. Rebot: Scaling robot learning with real-to-sim-to-real robotic video synthesis. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11351–11358. IEEE, 2025.
- Yunhai Feng, Natalie Leung, Jiaxuan Wang, Lujie Yang, Haozhi Qi, and Preston Culbertson. A minimalist retargeting-guided reinforcement learning recipe for dexterous manipulation. *arXiv preprint arXiv:2607.11874*, 2026.
- Harry Freeman, Chung Hee Kim, and George Kantor. Warped: Wrist-aligned rendering for robot policy learning from egocentric human demonstrations. *arXiv preprint arXiv:2604.10809*, 2026.
- Kai Fukazawa, Kunal Mundada, and Iman Soltani. Ramac: Multimodal risk-aware offline reinforcement learning and the role of behavior regularization. *arXiv preprint arXiv:2510.02695*, 2025.
- Joshua S Furtado, Hugh HT Liu, Gilbert Lai, Herve Lacheray, and Jason Desouza-Coelho. Comparative analysis of optitrack motion capture systems. In *Advances in Motion Sensing and Control for Robotic Applications: Selected Papers from the Symposium on Mechatronics, Robotics, and Control (SMRC’18)-CSME International Congress 2018, May 27-30, 2018 Toronto, Canada*, pp. 15–31. Springer, 2019.
- Shubham Goel, Georgios Pavlakos, Jathushan Rajasegaran, Angjoo Kanazawa, and Jitendra Malik. Humans in 4D: Reconstructing and tracking humans with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 14737–14748. IEEE, 2023.
- Ankur Handa, Karl Van Wyk, Wei Yang, Jacky Liang, Yu-Wei Chao, Qian Wan, Stan Birchfield, Nathan Ratliff, and Dieter Fox. Dexpivot: Vision-based teleoperation of dexterous robotic hand-arm system. In *2020 IEEE international conference on robotics and automation (ICRA)*, pp. 9164–9170. IEEE, 2020.
- Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- Ryan Hoque, Peide Huang, David Yoon, Mouli sivapurapu, and Jian Zhang. Egodex: Learning dexterous manipulation from large-scale egocentric video. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 4218–4237, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/07fcc6e2b89439d3ee5ab60939aaa6a0-Paper-Conference.pdf.
- Zhenyu Jiang, Cheng-Chun Hsu, and Yuke Zhu. Ditto: Building digital twins of articulated objects from interaction. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5606–5616. IEEE, 2022.

- Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Jim Fan, and Yuke Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16923–16930. IEEE, 2025.
- Vince Kurtz and Joel W Burdick. Generative predictive control: Flow matching policies for dynamic and difficult-to-demonstrate tasks. *arXiv preprint arXiv:2502.13406*, 2025.
- Andrew Lee, Ian Chuang, Dechen Gao, Kai Fukazawa, and Iman Soltani. Gaze on the prize: Shaping visual attention with return-guided contrastive learning. *arXiv preprint arXiv:2510.08442*, 2025.
- Jinhan Li, Yifeng Zhu, Yuqi Xie, Zhenyu Jiang, Mingyo Seo, Georgios Pavlakos, and Yuke Zhu. Okami: Teaching humanoid robots manipulation skills through single video imitation. *arXiv preprint arXiv:2410.11792*, 2024.
- Kailin Li, Puhao Li, Tengyu Liu, Yuyang Li, and Siyuan Huang. Maniptrans: Efficient dexterous bimanual manipulation transfer via residual learning. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6991–7003. IEEE, 2025.
- Wenzhao Lian, Tim Kelch, Dirk Holz, Adam Norton, and Stefan Schaal. Benchmarking off-the-shelf solutions to robotic assembly tasks. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1046–1053. IEEE, 2021.
- Qiayuan Liao, Takara E Truong, Xiaoyu Huang, Yuman Gao, Guy Tevet, Koushil Sreenath, and C Karen Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *Science Robotics*, 11(117):eadx8924, 2026.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- Vincent Liu, Ademi Adeniji, Haotian Zhan, Siddhant Haldar, Raunaq Bhirangi, Pieter Abbeel, and Lerrel Pinto. Egozero: Robot learning from smart glasses, 2025a. URL <https://arxiv.org/abs/2505.20290>.
- Xueyi Liu, Jianibieke Adalibieke, Qianwei Han, Yuzhe Qin, and Li Yi. Dextrack: Towards generalizable neural tracking control for dexterous manipulation from human references, 2025b. URL <https://arxiv.org/abs/2502.09614>.
- Juncheng Ma, Jianxin Bi, Yufan Deng, Xuanran Zhai, Kewei Zhang, Ye Huang, Bo Liang, Shukai Gong, Jiankai Tu, Xiaotian Tang, et al. Humanscale: Egocentric human video can outperform real-robot data for embodied pretraining. *arXiv preprint arXiv:2606.20521*, 2026.
- Robert Jomar Malate, Erik Bauer, Norica Bacuieti, Stefanos Charalambous, Elvis Nava, Robert K Katzschmann, and Benedek Forrai. Smooth operator: A real-time sampling-based algorithm for kinematic hand retargeting. *arXiv preprint arXiv:2607.07491*, 2026.
- Zhao Mandi, Yifan Hou, Dieter Fox, Yashraj Narang, Ajay Mandlekar, and Shuran Song. Dexmachina: Functional retargeting for bimanual dexterous manipulation, 2025. URL <https://arxiv.org/abs/2505.24853>.
- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903. IEEE, 2024.
- Bhawna Paliwal, Haritheja Etukuru, William Liang, Pieter Abbeel, Nur Muhammad Mahi Shafullah, and Jitendra Malik. Do as I do: Dexterous manipulation data from everyday human videos. *arXiv preprint arXiv:2606.19333*, 2026.
- Chaoyi Pan, Changhao Wang, Haozhi Qi, Zixi Liu, Homanga Bharadhwaj, Akash Sharma, Tingfan Wu, Guanya Shi, Jitendra Malik, and Francois Hogan. Spider: Scalable physics-informed dexterous retargeting, 2025. URL <https://arxiv.org/abs/2511.09484>.

- Georgios Papagiannis, Norman Di Palo, Pietro Vitiello, and Edward Johns. R+ x: Retrieval and execution from everyday human videos. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8284–8290. IEEE, 2025.
- Sungjae Park, Homanga Bharadhwaj, and Shubham Tulsiani. Demodiffusion: One-shot human imitation using pre-trained diffusion policy. *arXiv preprint arXiv:2506.20668*, 2025.
- Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In *Conference on Robot Learning*, pp. 1049–1065. PMLR, 2021.
- Yuzhe Qin, Yueh-Hua Wu, Shaowei Liu, Hanwen Jiang, Ruihan Yang, Yang Fu, and Xiaolong Wang. Dexmv: Imitation learning for dexterous manipulation from human videos. In *European Conference on Computer Vision*, pp. 570–587. Springer, 2022.
- Yuzhe Qin, Wei Yang, Binghao Huang, Karl Van Wyk, Hao Su, Xiaolong Wang, Yu-Wei Chao, and Dieter Fox. Anyteleop: A general vision-based dexterous robot arm-hand teleoperation system. *arXiv preprint arXiv:2307.04577*, 2023.
- Bin Qiu, Yanming Shao, Guanyu Cai, and Yao Mu. Enforcing human-like kinematics in dexterous piano playing via adversarial posture regularization. *arXiv preprint arXiv:2606.23848*, 2026.
- Robert N Rohling and John M Hollerbach. Optimized fingertip mapping for teleoperation of dextrous robot hands. In *[1993] Proceedings IEEE International Conference on Robotics and Automation*, pp. 769–775. IEEE, 1993.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pp. 234–241. Springer, 2015.
- Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635. JMLR Workshop and Conference Proceedings, 2011.
- Kenneth Shaw, Shikhar Bahl, and Deepak Pathak. Videodex: Learning dexterity from internet videos. In *Conference on Robot Learning*, pp. 654–665. PMLR, 2023.
- Zhenhao Shen, Jiaqi Liang, Jasper Lu, Feng Jiang, Yuran Wang, Chuanbo Wei, Jiayi Liu, Jianchun Yang, Qize Yu, Jiadi You, et al. Ld4wam: Learning latent dynamics from human videos for world action models. *arXiv preprint arXiv:2608.22403*, 2026.
- Chen Tang, Ben Abbatematteo, Jiaheng Hu, Rohan Chandra, Roberto Martín-Martín, and Peter Stone. Deep reinforcement learning for robotics: A survey of real-world successes. *Annual Review of Control, Robotics, and Autonomous Systems*, 8(1):153–188, 2025.
- The Mochi Team. An implicit physics engine for contact-rich simulation, 2026.
- Marcel Torne, Anthony Simeonov, Zechu Li, April Chan, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024.
- Chen Wang, Linxi Fan, Jiankai Sun, Ruohan Zhang, Li Fei-Fei, Danfei Xu, Yuke Zhu, and Anima Anandkumar. Mimicplay: Long-horizon imitation learning by watching human play. *arXiv preprint arXiv:2302.12422*, 2023.
- Hongwu Wang, Chenhao Yu, Youhao Hu, Jiachen Zhang, Yuanyuan Li, and Shiqi Luo. Bifrostumi: Bridging robot-free demonstrations and humanoid whole-body manipulation. *arXiv preprint arXiv:2605.03452*, 2026a.

- Zhi Wang, Botao He, Kelin Yu, Seungjae Lee, Ruohan Gao, Furong Huang, and Yiannis Aloimonos. Humanego: Zero-shot robot learning from minutes of human egocentric videos. *arXiv preprint arXiv:2605.24934*, 2026b.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M Rehg, Byron Boots, and Evangelos A Theodorou. Information theoretic mpc for model-based reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 1714–1721. IEEE, 2017.
- Jielin Wu, Shenzhe Yao, Guanqi He, Xiaohan Liu, Zhaoqing Zeng, Xiangrui Jiang, Han Yang, Wentao Zhang, and Hang Zhao. Toporetarg: Interaction-preserving retargeting for dexterous manipulation. *arXiv preprint arXiv:2606.16272*, 2026a.
- Wei Wu, Fangjing Wang, Fan Lu, He Sun, Shi Liu, Yunnan Wang, Yibin Yan, Yong Wang, Shuailei Ma, Xinyang Wang, et al. From foundation to application: Improving vla models in practice. *arXiv preprint arXiv:2607.06403*, 2026b.
- Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler, faster, stronger. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4840–4851. IEEE, 2024.
- Mengda Xu, Han Zhang, Yifan Hou, Zhenjia Xu, Linxi Fan, Manuela Veloso, and Shuran Song. Dexumi: Using human hand as the universal manipulation interface for dexterous manipulation. *arXiv preprint arXiv:2505.21864*, 2025a.
- Sirui Xu, Yu-Wei Chao, Liuyu Bian, Arsalan Mousavian, Yu-Xiong Wang, Liangyan Gui, and Wei Yang. Dexlore: Scalable neural control for dexterous manipulation from reference scoped exploration. In *Conference on Robot Learning*, pp. 2184–2199. PMLR, 2025b.
- Xinyue Xu, Jieqiang Sun, Jing, Dai, Siyuan Chen, Lanjie Ma, Ke Sun, Bin Zhao, Jianbo Yuan, Sheng Yi, Haohua Zhu, and Yiwen Lu. Dexcanvas: Bridging human demonstrations and robot learning for dexterous manipulation, 2025c. URL <https://arxiv.org/abs/2510.15786>.
- Haoru Xue, Chaoyi Pan, Zeji Yi, Guannan Qu, and Guanya Shi. Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing, 2024. URL <https://arxiv.org/abs/2409.15610>.
- Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models: A comprehensive survey of methods and applications. *ACM computing surveys*, 56(4):1–39, 2023.
- Yue Yang, Linfeng Zhao, Mingyu Ding, Gedas Bertasius, and Daniel Szafrir. Boss: Benchmark for observation space shift in long-horizon task. *IEEE Robotics and Automation Letters*, 10(9): 8882–8889, 2025.
- Zhao-Heng Yin, Changhao Wang, Luis Pineda, Krishna Bodduluri, Tingfan Wu, Pieter Abbeel, and Mustafa Mukadam. Geometric retargeting: A principled, ultrafast neural hand retargeting algorithm. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 17376–17382. IEEE, 2025.
- Yuhong Zhang, Zihan Gao, Shengpeng Li, Ling-Hao Chen, Kaisheng Liu, Runqing Cheng, Xiao Lin, Junjia Liu, Zhuoheng Li, Jingyi Feng, et al. Robowheel: A data engine from real-world human demonstrations for cross-embodiment robotic learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6664–6674, 2026.
- Weiguang Zhao, Tian Liang, Xihao Guo, Rui Zhang, Irwin King, and Kaizhu Huang. Towards robotic dexterous hand intelligence: A survey. *arXiv preprint arXiv:2605.13925*, 2026.
- Ruijie Zheng, Dantong Niu, Yuqi Xie, Jing Wang, Mengda Xu, Yunfan Jiang, Fernando Castañeda, Fengyuan Hu, You Liang Tan, Letian Fu, et al. Egoscale: Scaling dexterous manipulation with diverse egocentric human data. *arXiv preprint arXiv:2602.16710*, 2026.

Kefei Zhu, Fengshuo Bai, YuanHao Xiang, Yishuai Cai, Xinglin Chen, Ruochong Li, Xingtao Wang, Hao Dong, Yaodong Yang, Xiaopeng Fan, et al. Dexflywheel: A scalable and self-improving data generation framework for dexterous manipulation. *Advances in Neural Information Processing Systems*, 38:4236–4265, 2025.

Xinghao Zhu, Zixi Liu, Shalin Jain, Chenran Li, Milad Noori, Michael Andres Lin, Huihua Zhao, John Welsh, Mrinal Verghese, Wei Liu, et al. Learning dexterous manipulation using contact wrench guidance from human demonstration. *arXiv preprint arXiv:2607.00033*, 2026.

Appendix

A	Extended Related Work	17
A.1	Comparing Retargeting Datasets	17
A.2	Dexterous Retargeting	17
B	Experimental Settings	18
B.1	GNR Architecture and Training	18
B.2	Evaluation	18
B.3	Optimizer Baselines	18
C	Data Engine Details	19
C.1	Real-to-Sim Transfer and Kinematic Retargeting	19
C.2	Skill Segmentation	20
C.3	Grasp Establishment	20
C.4	Contact-Force Labels	20
C.5	Geometry Variants	20
D	HOT3D	21
D.1	Data Generation	21
D.2	Training & Test Set	22
D.3	Evaluation	22
D.4	Dataset Expansion	24
D.5	Trajectory Analysis	24
E	ShapeFilter	24
E.1	Settings	24
E.2	Evaluation	25
E.3	Baselines	26
E.4	Ablations	26
E.5	Efficiency	27
E.6	Trajectory Analysis	28
F	NIST	28
F.1	Settings	28
F.2	Optimization and Efficiency	28
G	Dataset Generation Efficiency	29
G.1	Compute Cost	29
G.2	Cost versus Dataset Coverage	30
H	Multi-Embodiment Evaluation	30
H.1	Setup and Results	30
H.2	Efficiency Comparison	31
I	Learning from Retargeted Demonstrations	31
I.1	Behavior Cloning	31
I.2	Data Flywheel	32
J	Retargeting Demos	33
J.1	ShapeFilter	33
J.2	NIST	34
J.3	HOT3D	36

A EXTENDED RELATED WORK

A.1 COMPARING RETARGETING DATASETS

Table 3 compares datasets used by recent retargeting works along the diversity and difficulty axes. As discussed in Section 1, the computational cost and engineering burden of physics-based retargeting limit scaling along both axes.

Along the diversity axis, most works evaluate on a few thousand trajectories with limited object geometries. We apply GNR within our real-to-sim data engine (Section 4) to scale both the number of trajectories and the range of geometries from egocentric videos in HOT3D (Banerjee et al., 2025). This produces 223k paired human and feasible robot trajectories.

Along the difficulty axis, existing works primarily evaluate on short-horizon tasks lasting a few seconds (Pan et al., 2025; Liu et al., 2025b), or use longer demonstrations lasting 10 to 30 seconds with success criteria that tolerate centimeter-level errors (Zhu et al., 2026; Mandi et al., 2025). Long-horizon tasks requiring millimeter-level precision remain underexplored in existing retargeting work. Such tasks require accurate human motion references, but motion recovered from egocentric videos may lack the precision needed for successful retargeting. We therefore use motion capture with a wearable exoskeleton to collect high-precision human demonstrations for ShapeFilter and NIST. These long-horizon, multi-stage tasks require millimeter-level precision and are, to our knowledge, among the *most challenging* tasks studied in dexterous retargeting.

Table 3: Recent physics-based retargeting methods on the two scaling axes: *diversity* (number of demonstrations and distinct geometries) and *difficulty* (task precision and average demonstration length).

Method	Demonstrations	Geometries	Precision (mm)	Horizon
DexTrack (Liu et al., 2025b)	3,585	257	100	short
AdaDexTrack (Adalibieke et al., 2026)	2,765	50	100	short
ManipTrans (Li et al., 2025)	3,300	1,200	30-80	short
DexMachina (Mandi et al., 2025)	7	5	10 to 90	short to long
SPIDER (Pan et al., 2025)	2,885	103	100	short
Do-As-I-Do (Paliwal et al., 2026)	500	N/A	100	short
DexCanvas (Xu et al., 2025c)	12,000	30	50	short to long
DexFlyWheel (Zhu et al., 2025)	2,000	80	20	short
CHORD (Zhu et al., 2026)	4,739	10	150	short to long
GNR (HOT3D)	223k	3.3k	150	short (2 s)
GNR (ShapeFilter)	3,037	1	1.96	long (11.0 s)
GNR (NIST)	358	1	1.0	long (29 s)

A.2 DEXTEROUS RETARGETING

Robot Learning from Human Data. Human video provides a scalable source of manipulation data (Shaw et al., 2023; Bharadhwaj et al., 2024; Papagiannis et al., 2025; Liu et al., 2025a; Ma et al., 2026). Robot systems increasingly learn from internet and egocentric video (Shaw et al., 2023; Wang et al., 2023; Li et al., 2024; Wang et al., 2026b; Freeman et al., 2026; Wang et al., 2026a; Zhao et al., 2026), but differences in morphology and actuation create an embodiment gap that prevents human motions from being transferred directly to robots (Chen et al., 2026a; Zhao et al., 2026; Wu et al., 2026b). Retargeting has emerged as a promising approach to transfer human motions to robots (Pan et al., 2025; Mandi et al., 2025; Dariush et al., 2008a).

Kinematic Retargeting. Kinematic approaches match human and robot fingertips or keypoints and recover robot configurations with IK (Dariush et al., 2008a; Rohling & Hollerbach, 1993; Antotsiou et al., 2018; Dariush et al., 2008b; Yin et al., 2025; Handa et al., 2020; Qin et al., 2023; Malate et al., 2026; Wu et al., 2026a). Source motion may come from motion capture systems, wearable hardware, or video (Bahety et al., 2024; 2025; Li et al., 2024; Park et al., 2025; Ma et al., 2026; Wu et al., 2026b). Existing works have explored object-aware retargeting (Li et al., 2024),

policy refinement for retargeting (Park et al., 2025), and visual synthesis to reduce geometric or visual mismatch (Freeman et al., 2026; Shen et al., 2026; Wu et al., 2026b), but kinematic constraints alone do not ensure dynamically feasible trajectories (Malate et al., 2026).

Physics-based Retargeting. Physics-based methods produce dynamically feasible trajectories through RL (Liu et al., 2025b; Zhu et al., 2026; Mandi et al., 2025; Feng et al., 2026; Adalibieke et al., 2026) or sampling-based control (Pan et al., 2025; Paliwal et al., 2026; Xue et al., 2024; Malate et al., 2026). RL can suffer from costly and unstable policy optimization and often requires careful reward engineering (Feng et al., 2026; Pan et al., 2025; Lee et al., 2025; Fukazawa et al., 2025). Sampling-based control avoids policy training but still optimizes each trajectory independently. GNR learns a generative model of dynamically feasible trajectories to improve sample-efficiency for retargeting new demonstrations. We deploy GNR open-loop, or use GNR to provide MPC with on-manifold initialization for contact-rich tasks. GNR generalizes to OOD demonstrations and solves demonstrations on which MPC fails, thus expanding the dataset while boosting efficiency.

B EXPERIMENTAL SETTINGS

B.1 GNR ARCHITECTURE AND TRAINING

The GNR model for HOT3D combines a Point Transformer V3 geometry encoder, taking in 1,024 points as inputs, with a temporal U-Net of widths (128, 256, 512). The geometry, conditioning, and flow-step embeddings have dimensions 96, 256, and 128, respectively. The model has 19.57M trainable parameters and uses ten Euler steps for inference. We train it on 28,480 episodes using bfloat16 mixed precision and batch size 128 on one NVIDIA A100.

The ShapeFilter and NIST models use temporal U-Net widths (128, 256, 512), kernel size 5, group normalization, and a 128-dimensional flow-step embedding. ShapeFilter conditions on the IK reference, arm joint state, and either the object pose or a geometry encoding from Point Transformer V3, using 64-frame sequences for training and inference. The main experiments use object poses. Geometry conditioning achieves similar performance and is reported in Table 11. NIST conditions on the reference and knob pose and uses complete episodes. We train ShapeFilter for 100 epochs and NIST for 200 epochs. Both use AdamW with a constant learning rate of 4×10^{-4} , weight decay 10^{-6} , gradient clipping at 1.0, EMA decay 0.999, batch size 128, and mixed precision on one GPU.

B.2 EVALUATION

We use S for the number of GNR samples per demonstration, n_s for samples per MPC optimizer iteration, and M for the maximum number of iterations in each optimization, giving a sample budget of $n_s M$. We compare different methods with the success criterion reported in Table 4.

Table 4: Success criteria of different benchmarks.

Benchmark	Success criterion
ShapeFilter	Object centroid lies in the insertion region $[-80, 80] \times [0, 160] \times [-80, 80]$ mm.
NIST	Horizontal error < 1 mm, screwing down by at least 8 mm, and axis error < 0.003 rad.
HOT3D	Object position error < 150 mm and orientation error $< 40^\circ$. Mean capsule penetration ≤ 5 mm. Median squeeze force after grasp release ≥ 1 N. Per-finger force 95th percentile ≤ 100 N.
SPIDER	Mean object-position error ≤ 0.10 m and mean object-orientation error ≤ 0.50 rad.

B.3 OPTIMIZER BASELINES

DIAL-MPC (Xue et al., 2024). DIAL-MPC perturbs the control sequence using annealed sampling noise, evaluates the resulting simulation rollouts, and updates the control sequence. Table 5 summarizes the configurations and hyperparameters used in the main experiments.

Table 5: DIAL-MPC settings for the main experiments. n_s denotes samples per iteration and M the maximum number of MPC iterations; Samples gives $n_s M$ for each phase.

Benchmark	Phase	n_s	M	Samples	Horizon	Stride	Active coordinates
ShapeFilter	insertion	500	6	3,000	20	5	arm and hand
	Exploitation fraction 0.15; exploitation noise 0.5; temperature 0.08; trajectory-noise decay 0.7.						
NIST	wrist alignment	86	10	860	7	2	end effector
	finger closure	86	10	860	6	1	thumb and index
	final refinement	172	10	1,720	13	4	arm and hand
	Initial and final noise scales 1.0; exploitation fraction 0.15; exploitation noise 0.5.						
HOT3D	tracking	300	10	3,000	full episode	full episode	hand
	Temperature 0.03; trajectory-noise decay 0.99; sampling noise scale decreases log-linearly from 0.10 to 0.04.						

MPPI and SPIDER. We evaluate MPPI (Williams et al., 2017) and SPIDER (Pan et al., 2025) on SPIDER’s multi-embodiment benchmark. We use SPIDER’s public implementation and configurations, which combine iterative physics optimization with annealed sampling noise and contact guidance. Both draw $n_s=1,024$ candidates per optimizer iteration and run up to $M=16$ MPC iterations on OakInk and $M=32$ on GigaHands, with early stopping. MPPI uses a fixed noise scale of 1.0, whereas SPIDER anneals it to 0.01. The results are reported in Section H.

CEM, iCEM, and CMA-ES. We evaluate CEM, iCEM, and CMA-ES as additional sampling-based baselines. On ShapeFilter, CEM uses 32 or 500 samples per iteration to evaluate test-time scaling; iCEM and CMA-ES use 32. All three run for 6 iterations with a 20-step planning horizon.

Table 6: Sampling-based optimizer settings.

Method	Benchmark	n_s	M	Samples	Sampling settings
MPPI	HOT3D	300	10	3,000	Softmax-weighted update using DIAL-MPC sampling noise settings.
CEM	ShapeFilter	32	6	192	Elite ratio 0.10; mean/variance smoothing 0.10; white Gaussian noise; fixed sample count; optimizer reset at the start of each optimization.
		500		3,000	
iCEM	ShapeFilter	32	6	192	Elite ratio 0.10; smoothing 0.10; colored-noise exponent 2.0. HOT3D: fixed sample count; reuse 30% of elite samples; reset the optimizer at the start of each optimization; return the best trajectory rather than the average of elite trajectories.
	HOT3D	300	10	3,000	
CMA-ES	ShapeFilter	32	6	192	Full covariance over optimized control dimensions. ShapeFilter: 29-dimensional control; initialized from the DIAL-MPC noise scale. HOT3D: hand control with the arm fixed; initial noise scale 0.5; relative noise scales from DIAL-MPC; simulate the selected trajectory once more (3,001 rollouts total).
	HOT3D	300	10	3,000	

C DATA ENGINE DETAILS

C.1 REAL-TO-SIM TRANSFER AND KINEMATIC RETARGETING

Capture to simulated human hand. The capture system and simulator represent joint motion differently and use different rest poses and relative bone lengths. We transfer motion through bone positions and orientations in a shared coordinate system. After transforming the captured motion into the simulator’s coordinates, we use forward kinematics to obtain 21 hand landmarks and construct coordinate frames for 19 bones. The simulator’s IK converts these targets into a 33-DoF hand configuration while preserving the simulated hand’s relative bone lengths. A fixed rotation aligns the rest poses. Mirroring is applied to both the recorded hand representation and the simulated hand model to align their handedness. We obtain joint positions from the hand landmarks rather than the bone-frame origins, which coincide with the wrist in the rest pose in this representation.

Simulated human hand to robot. The DG-5F’s limited thumb motion across the palm and rigid palm prevent it from reproducing all human hand poses. We use the wrist pose and five fingertip positions as IK targets, initializing each frame with the previous solution to encourage smooth motion.

Our simulator uses a right-handed DG-5F model, so we mirror the complete scene across the sagittal plane for left-handed segments. The resulting robot configurations and recorded object poses form the kinematic reference $\tilde{x}_{0:T}$.

C.2 SKILL SEGMENTATION

We divide multi-skill recordings into single-skill segments before retargeting; individually recorded single-skill demonstrations are used directly.

Contact heuristic. When 3D hand and object states are available, as in HOT3D, we estimate hand-object contact from their relative positions. A segment is an uninterrupted sequence of frames in which at least two fingers lie within 20 mm of the object surface, measured using a k-d tree to search the hand landmarks obtained by forward kinematics. A segment ends if the object remains set down for more than 5 s. Gaps shorter than 0.5 s are merged, and very short segments are discarded. Each segment starts at first contact and extends past release. Segments are labeled left-handed, right-handed, or two-handed. Applied to HOT3D, this yields roughly 5,000 segments over 33 objects: about 40% two-handed, 40% right-handed, and 20% left-handed. The subset used by the data engine is reported in Table 8.

Vision-language segmenter. When only RGB video is available, we use a vision-language model (VLM) to identify hand-object contact in sampled frames. We group consecutive frames by contact state, discard isolated single-frame changes, and merge gaps shorter than 0.7 s. The VLM then reviews sampled frames from each segment to correct which hand and object are involved. Otherwise, we detect contact using the 3D hand and object positions as discussed in the contact heuristic method above.

C.3 GRASP ESTABLISHMENT

Segments may begin with the object already grasped. IK aligns the fingertips but can leave the DG-5F palm or knuckles intersecting the object. This initial penetration can eject the object or trap it against the hand, producing low tracking error without a valid grasp.

We therefore re-establish the grasp in simulation. The hand starts open and closes around the object while the object is fixed in place with gravity disabled. After roughly 12 frames, the constraint is gradually released and gravity is restored. Force and penetration are evaluated after the constraint is released.

C.4 CONTACT-FORCE LABELS

We use a signed distance field to represent the object’s collision geometry. A penalty contact model computes the normal force from penetration depth; the field gradient gives the surface normal. We execute the optimized control sequence open loop with the same proportional-derivative (PD) controller, recording the position, normal, penetration depth, and force at every contact point.

Contact stiffness affects the magnitude of the recorded forces. We use a value one tenth of the simulator default, with lower finger joint controller gains.

We close holes in object meshes to make them watertight, so the signed distance field has a well-defined inside and outside. Physics runs at a higher rate than control to reduce penetration between control updates. A short temporal median filter, applied per finger and force component, removes isolated outliers.

C.5 GEOMETRY VARIANTS

Physics augmentation varies object density and surface friction; geometry augmentation modifies the object mesh.

Table 7 lists the five geometry augmentations. Deformation and stretching preserve the correspondence between vertices in the original and modified meshes, allowing grasp contact points to transfer to the variant. The other three regenerate the mesh from a reference image with SAM3D (Chen et al.,

2026b) and do not preserve this correspondence. Together, they define 200 variants per object, or 6,600 possible meshes across the 33 source objects.

We replace the object mesh in a successfully retargeted scene, then re-optimize the control sequence and evaluate it with the same success criterion Γ . Sampling 10 geometry variants for each of 728 source motions, with 24 random optimizer seeds per pair, gives 174,720 trials. The 7,280 motion-variant pairs involve 3,976 distinct meshes, since motions involving the same object sample from the same set of available geometry variants. Of these, 2,723 yield at least one successful trajectory. Together with the 33 original meshes, they give the 2,756 geometries in the original MPC-success dataset.

Table 7: Geometry augmentations and variants per object.

Edit	Implementation	Variants per object
Regeneration	Regenerate from the reference image with a new random seed	20
Deformation	Deform the mesh by different amounts along each axis	60
Occlusion completion	Mask part of the reference image and complete the shape	20
Rotation	Rotate about the vertical axis, then regenerate	20
Stretch	Scale the original mesh along a single axis	80
Total		200

D HOT3D

D.1 DATA GENERATION

Egocentric hand-object recordings are retargeted to the DG-5F hand with 26-dimensional joint control. The data engine augments 780 human motions from 210 recordings and 33 objects by varying pose, timing, rotation, scale, friction, and geometry. Among the 360,150 evaluated demonstrations, DIAL-MPC succeeds on 95,344 (26.47%). GNR solves another 127,136 MPC-failure demonstrations, giving 222,480 successful trajectories within this population (Table 8).

Table 8: HOT3D statistics for the 360,150 evaluated demonstrations. Columns separate demonstrations using original and modified object geometries; original geometries are augmented in pose, timing, mass, and friction. Trajectory counts and distinct geometry counts are reported separately.

Quantity	Original geometries	Modified geometries	Total
Evaluated demonstrations	186,310	173,840	360,150
DIAL-MPC successes	47,426	47,918	95,344
DIAL-MPC failures	138,884	125,922	264,806
Additional GNR successes ($S=256$)	74,391	52,745	127,136
MPC or GNR successes	121,817	100,663	222,480
Source motions	780	728	780
Distinct geometries	33	3,976	4,009
Geometries with MPC successes	33	2,722	2,755
Geometries with MPC or GNR successes	33	3,265	3,298

Augmentation. During the initial search, the data engine evaluates the original scene and 16 perturbations of object translation, start time, scale, rotation, and friction. Translation is shifted by 15 mm in either direction along the palm normal or grasp axis; time is shifted by -5 , -3 , $+3$, or $+5$ frames; scale is multiplied by 1.15 or 1.30; rotation is shifted by 10° in either direction about the up or grasp axis; and friction is multiplied by 0.5 or 2.0. Geometry augmentation adds reconstructed shapes, deformations, shapes completed from partially occluded images, rotations about the vertical axis, and stretches. Every variant is re-optimized and re-evaluated under physics.

Success criterion. Success uses a position-error threshold of 150 mm (Table 4) following CHORD (Zhu et al., 2026), rotation error below 40° , mean capsule penetration at most 5 mm, median squeeze force after grasp release at least 1 N, and per-finger 95th-percentile force at most 100 N.

D.2 TRAINING & TEST SET

GNR is trained on 28,480 DIAL-MPC successes from pose, timing, mass, and friction augmentations. The training set contains 1,838,731 frames (17.03 hours). Each demonstration has about 64 frames (2.15 s at 30 Hz). The dataset covers 23 objects and 353 source motions.

Because sampling-based optimizers are costly, we randomly sample a test set with 1,000 demonstrations and compare GNR with other sampling-based optimizers on this set. It comprises 203 OOD-object, 338 OOD-grasp, and 459 OOD-geometry demonstrations. GNR results are reported in Figure 3. Beyond this test set, we also apply GNR to the 360k full dataset. Results are reported in Section D.4.

D.3 EVALUATION

D.3.1 TEST SET

The HOT3D dataset contains 33 scanned objects in total. GNR used in Section 5 is trained on 23 objects and 353 source motions. The 1,000 test demonstrations are divided into three disjoint subsets: 203 with unseen objects, 338 with unseen grasps, and 459 with unseen geometries.

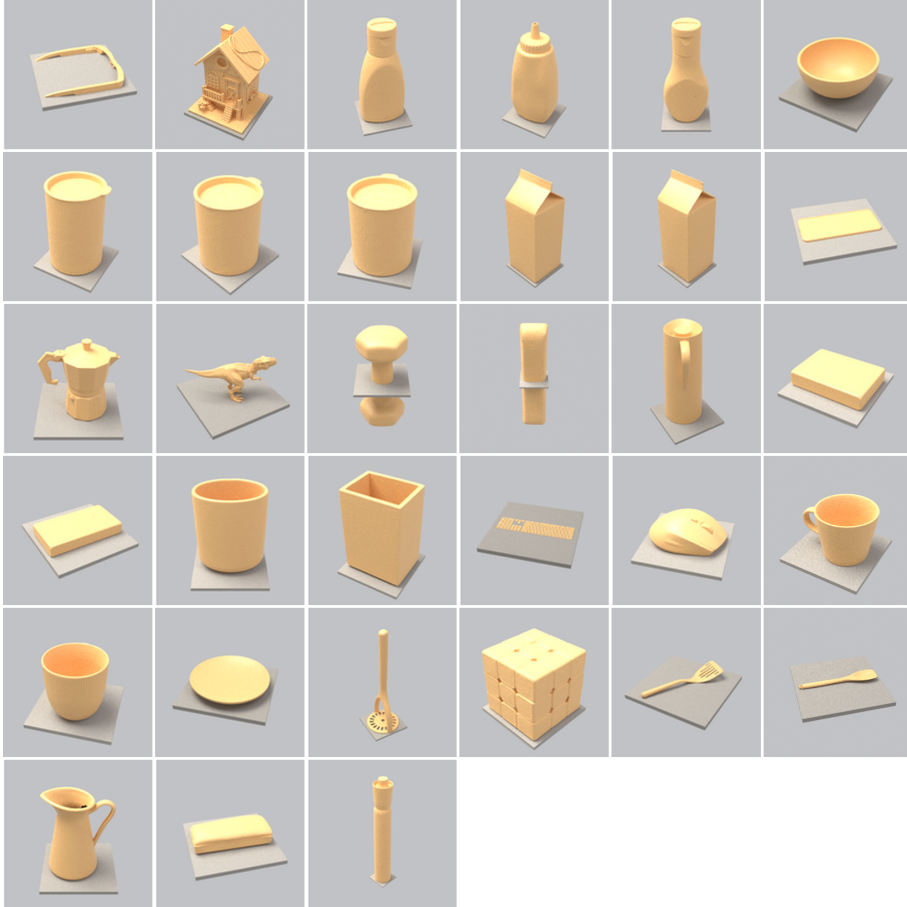


Figure 6: Objects in HOT3D.



Figure 7: Examples of geometry variants. Each row uses the same original object.

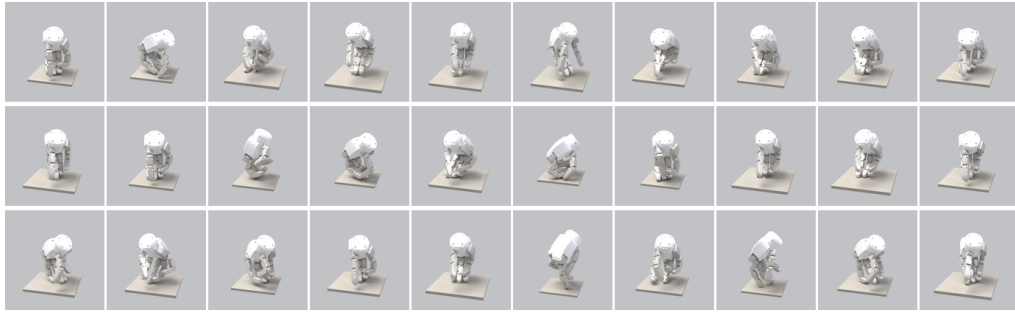


Figure 8: DG-5F grasp configurations at a shared camera view and scale.

On the test set, GNR success rises from 12.81% to 44.33% on OOD objects, from 10.06% to 42.90% on OOD grasps, and from 26.36% to 71.24% on OOD geometry as S increases from 1 to 256.

D.4 DATASET EXPANSION

We apply GNR to 264k MPC-failure demonstrations. For this expansion, GNR is trained on 28,480 MPC successes. A demonstration succeeds if any of its S candidates succeeds; sampling stops upon success.

GNR solves 19,384 MPC-failure demonstrations (7.32%) at $S=1$, increasing to 101,129 (38.19%) at $S=64$, 115,017 (43.43%) at $S=128$, and 127,136 (48.01%) at $S=256$ (Figure 9). Combining these additional trajectories with the 95k MPC successes yields 222,806 feasible trajectories (223k). Geometry coverage grows from 2,756 to 3,298 (3.3k), including 542 new geometries.

We also evaluate the same model on 95k MPC-success demonstrations. SR increases from 43.22% at $S=1$ to 84.75% at $S=64$ and 89.30% at $S=256$ (85,143 successes). Among the demonstrations outside the training set, SR rises from 40.68% to 88.91%. Across the entire 360k demonstrations, GNR solves 212,279 (58.94%) at $S=256$.

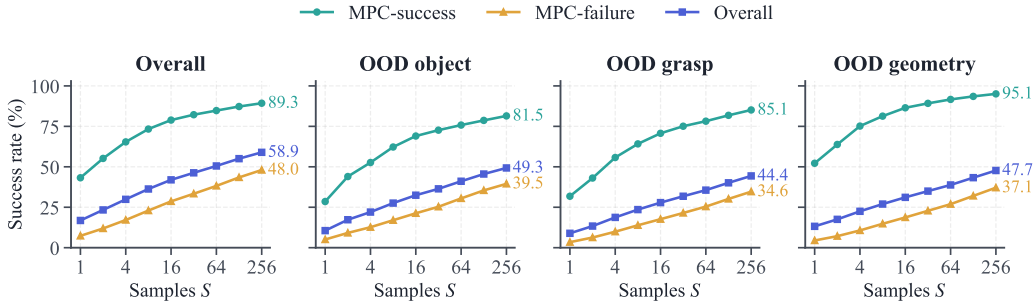


Figure 9: Test-time scaling on 360k HOT3D demonstrations: 95k MPC-success and 264k MPC-failure demonstrations. The curves show GNR’s SR across three OOD subsets.

D.5 TRAJECTORY ANALYSIS

Figure 10 compares object and finger-joint tracking errors and finger-control jerk.

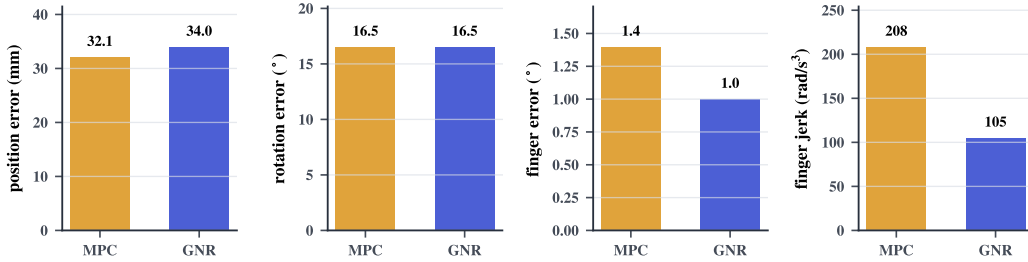


Figure 10: HOT3D trajectory quality. From left to right: median object-position, object-orientation, and finger-joint angular errors on the randomly sampled 1,000-demonstration test set, followed by median finger-control jerk. GNR approximately halves jerk while preserving comparable object tracking.

E SHAPEFILTER

E.1 SETTINGS

Task. A Tesollo DG-5F hand on a Franka FR3 grasps a cross-shaped object and inserts it into a matching hole on the box lid. The 29-D action comprises end-effector translation, a continuous 6-D rotation, and 20 hand joints. Control runs at 20 Hz and physics at 60 Hz.

Dataset. Pose augmentation of 605 human demonstrations produces 3,037 demonstrations and 665,770 frames. Unless otherwise stated, we use the motion split with 2,448 training and 589 test demonstrations from 484 and 121 underlying human demonstrations, respectively.

Success criterion. The object centroid enters the insertion region $[-0.08, 0.08] \times [0, 0.16] \times [-0.08, 0.08]$ m in the box coordinate frame, indicating insertion completion. The insertion tolerance is 1.96 mm.

Domain randomization. We apply perturbations to object poses per demonstration for domain randomization. We randomize object and scene poses over the ranges in Table 9. The box and lid move rigidly together.

Table 9: ShapeFilter domain-randomization settings.

Scene element	Perturbation	Sampling range
Cross-shaped object	horizontal position	$[-30, 30]$ mm per axis
Cross-shaped object	yaw	$[-10, 10]^\circ$
Box and lid	horizontal position	$[-30, 30]$ mm per axis
Box and lid	yaw	$[-2.5, 2.5]^\circ$
Complete scene	vertical position	$[-20, 20]$ mm

Captured demonstrations add variation beyond these ranges. For example, initial cross positions span approximately ± 167 mm and ± 121 mm along the two table axes.

E.2 EVALUATION

We evaluate GNR’s generalization to spatial OOD demonstrations by creating three different spatial OOD training-test splits. The splits assign demonstrations by initial object position in the four quadrants of the table as shown in Figure 11. We create the splits using 1, 2, or 4 quadrants for training, and the remaining quadrants for testing.

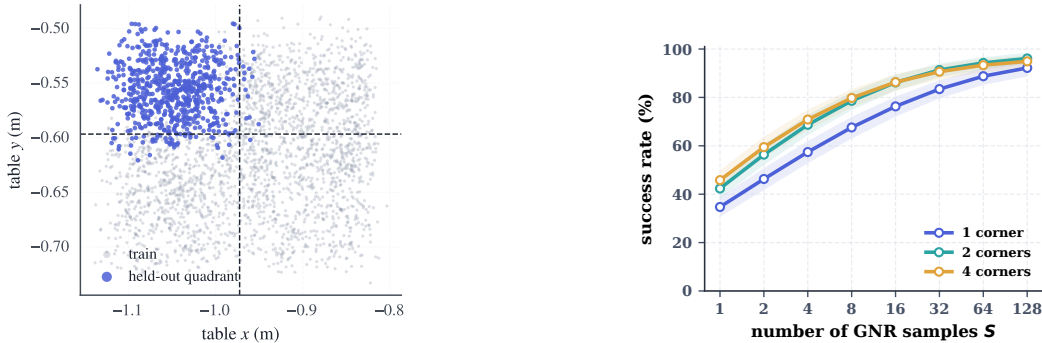


Figure 11: ShapeFilter spatial generalization. The training (gray) and test (blue) quadrants are shown at left. At right, we show the test-time scaling curves of GNR on the test split.

E.3 BASELINES

Table 10: ShapeFilter success rates. Optimizer budgets are samples per iteration $n_s \times \text{MPC}$ iterations M .

Method	$n_s M$ or S	Success (%)
DIAL-MPC	32×6	48.4
DIAL-MPC	500×6	92.9
CEM	32×6	50.0
CEM	500×6	82.8
iCEM	32×6	65.6
CMA-ES	32×6	40.6
GNR	8	87.23
GNR	16	92.26
GNR	32	95.44
GNR	64	97.37

E.4 ABLATIONS

Conditioning. We compare three conditioning inputs. State uses the IK reference and simulator object state. Geometry uses a 1,024-point cloud as the condition. Geometry + state provides both.

Table 11: ShapeFilter conditioning ablation. We report success rates (%) across different numbers of GNR samples S using different conditioning.

Samples S	State	Geometry	Geometry + state
1	50.42	50.93	53.48
2	67.06	66.38	69.10
4	78.44	80.31	78.61
8	87.10	88.62	86.59
16	92.36	93.72	94.06
32	95.93	96.77	97.11
64	97.45	98.13	98.81
128	98.47	98.81	99.15
256	99.32	99.49	99.32

Residual prediction. We compare absolute and residual prediction on ShapeFilter with one, two, or four training corners.

With one, two, and four training corners, residual prediction reaches 34.72%, 42.35%, and 45.82% success at $S=1$, and 92.19%, 96.09%, and 94.92% at $S=128$ (Figure 12), showing that the residual prediction is beneficial especially when S is small.

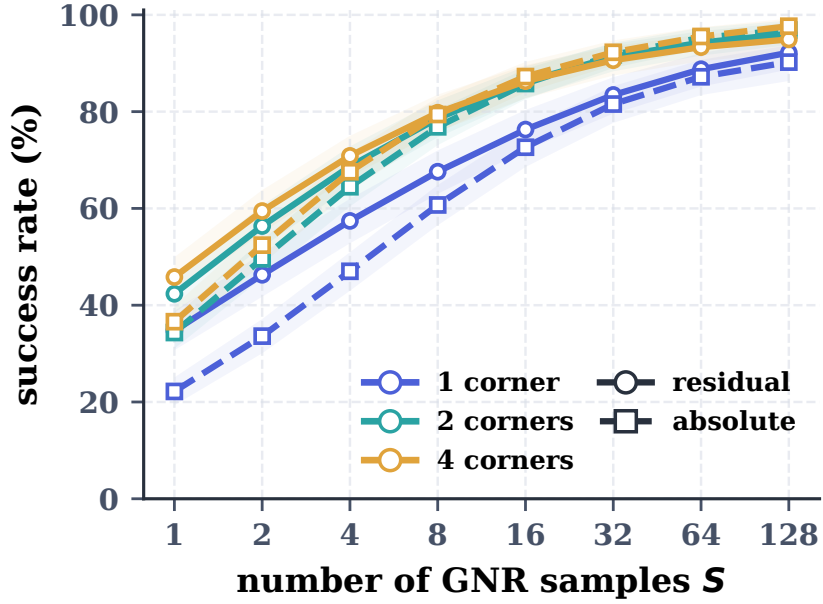


Figure 12: Performance of GNR using either residual or absolute prediction with one, two, or four training corners. Shadows show 95% confidence intervals.

E.5 EFFICIENCY

Tables 12 and 15 report success and generation cost. Figure 13 compares total generation cost as dataset size grows; App. J.1 shows qualitative comparisons.

Table 12: `ShapeFilter` dataset-generation efficiency. Dataset success rates include the successful seeds and apply the test-set success rate to the remaining demonstrations. Costs include MPC seed generation, GNR inference, and simulation, in thousands of core-h.

Seed set size N	Quantity	Samples S								
		1	2	4	8	16	32	64	128	256
50	Success (%)	13.0	17.2	33.4	47.4	66.4	83.1	93.2	96.7	99.5
	Cost (k core-h)	0.235	0.263	0.313	0.390	0.501	0.627	0.740	0.832	0.890
100	Success (%)	21.0	30.5	47.5	66.7	81.3	91.1	96.4	98.7	99.3
	Cost (k core-h)	0.438	0.463	0.504	0.560	0.626	0.688	0.745	0.789	0.828
200	Success (%)	33.4	44.3	57.7	71.9	80.0	89.1	94.9	96.8	99.0
	Cost (k core-h)	0.843	0.865	0.898	0.944	1.005	1.081	1.163	1.243	1.311
500	Success (%)	48.2	58.4	68.8	77.3	84.1	89.8	94.0	96.6	97.6
	Cost (k core-h)	2.061	2.077	2.102	2.137	2.186	2.251	2.330	2.417	2.526
1,000	Success (%)	62.2	70.1	80.0	84.9	90.9	94.4	96.8	98.2	98.7
	Cost (k core-h)	4.089	4.101	4.118	4.141	4.172	4.209	4.252	4.299	4.362
2,448	Success (%)	90.8	93.8	96.0	97.5	98.5	99.1	99.5	99.7	99.9
	Cost (k core-h)	9.964	9.967	9.970	9.974	9.979	9.985	9.992	10.000	10.008

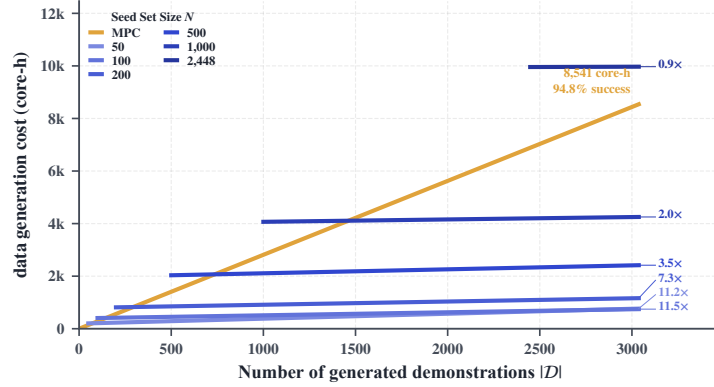


Figure 13: ShapeFilter generation cost versus dataset size. MPC generates all $|\mathcal{D}|$ demonstrations; GNR uses MPC for N successful seeds and generates the remaining $|\mathcal{D}| - N$ with configurations that meet or exceed the best MPC success rate. Numbers show the reduction in total cost.

E.6 TRAJECTORY ANALYSIS

Figure 14 compares tracking error and smoothness.

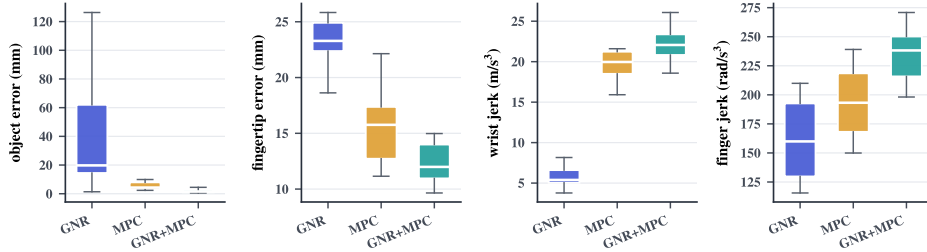


Figure 14: ShapeFilter trajectory errors and jerk comparison. GNR has lower jerk; GNR+MPC has the lowest object and fingertip tracking errors.

F NIST

F.1 SETTINGS

Task. In a bimanual Franka/DG-5F system, one arm grasps and screws an M8 knob onto a threaded stud. The 29-D action contains translation, continuous 6-D rotation, and 20 hand joints.

Dataset. The dataset contains 358 successful demonstrations and 206,946 frames. At 20 Hz, the mean trajectory duration is 29 s.

Success criterion. Success requires horizontal error below 1 mm, screwing down by at least 8 mm, and axis error below 0.003 rad.

F.2 OPTIMIZATION AND EFFICIENCY

Optimization has three phases. Wrist alignment uses 86 samples for 10 iterations over seven steps at stride two. Finger closure uses 86 samples for 10 iterations over six steps. The final stage of screwing is the hardest and uses 172 samples for 10 iterations over 13 steps at a stride of 4 and 200 Hz physics, using approximately 83% of the compute. NIST MPC sample budgets sum all three stages and have 3,440 samples in total.

The first two stages contribute 37.28 core-h per demonstration. The complete three-stage optimization costs 221.05 core-h (6.32 wall-hours) for MPC and 73.94 core-h (about 2.11 wall-hours) for GNR+MPC in Table 2, including GNR inference. Qualitative results are in App. J.2. Table 13 lists the configurations used for cost calculation; Figure 15 compares methods at similar dataset success rates. Dataset-level totals are in Table 15 (App. G).

Table 13: NIST dataset-generation efficiency. Dataset success rates include the successful seeds and estimated successes on the remaining demonstrations. Each MPC budget uses the schedule with the highest success rate. GNR generation costs include MPC seed generation, GNR inference, and GNR-initialized MPC, in thousands of CPU core-h (App. F.2).

$N=100$			$N=200$			$N=294$		
MPC samples	Success (%)	Cost (k core-h)	MPC samples	Success (%)	Cost (k core-h)	MPC samples	Success (%)	Cost (k core-h)
1,736	64.0	35.90	1,736	79.3	57.34	1,736	91.9	77.41
1,752	70.7	36.79	1,752	83.4	57.89	1,752	93.3	77.61
1,768	74.1	37.42	1,784	89.0	58.91	1,784	95.5	77.97
1,784	79.7	38.17	1,816	90.3	59.08	1,816	97.2	78.37
1,816	80.9	39.81	1,848	93.8	60.10	1,912	97.8	78.95
1,848	85.4	41.02	2,104	95.9	64.46	—	—	—
1,912	87.6	42.51	—	—	—	—	—	—
1,976	92.1	44.52	—	—	—	—	—	—
2,104	93.2	45.91	—	—	—	—	—	—

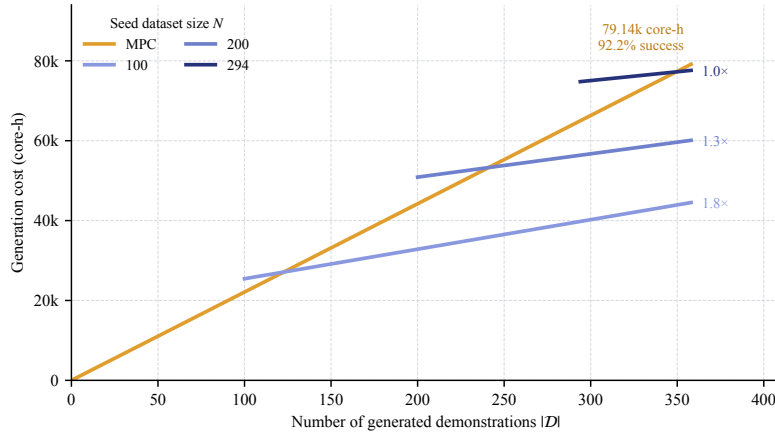


Figure 15: NIST generation cost versus dataset size. MPC generates all $|\mathcal{D}|$ demonstrations; GNR+MPC uses MPC for N successful seeds and generates the remaining $|\mathcal{D}| - N$ using evaluated configurations with similar dataset success rates. Numbers show the reduction in total cost.

G DATASET GENERATION EFFICIENCY

G.1 COMPUTE COST

We report both wall time and CPU compute time (core-h).

Table 14: Measured per-episode generation cost.

Task	Method	Sample budget	Core-h / episode	Wall h / episode
HOT3D	DIAL-MPC	3,000	1.0777	0.2427
HOT3D	iCEM	3,000	1.4213	0.3282
HOT3D	CMA-ES	3,000	1.2272	0.2877
HOT3D	GNR, $S=256$	256	0.1094	0.0383
ShapeFilter	MPC	3,000	3.2001	0.0472

GPU training time. On one H100 GPU, ShapeFilter training takes 0.0108 to 0.5022 GPU-h over 100 epochs as the seed set grows from 50 demonstrations to the full training set. NIST training takes 0.0202 to 0.0406 GPU-h over 200 epochs with 100 to 294 demonstrations. The training cost is overall marginal compared to physics-based sampling.

G.2 COST VERSUS DATASET COVERAGE

Table 15 sums CPU generation costs for MPC seed generation and GNR generation. HOT3D compares full MPC generation, dataset expansion, and generation from 28k MPC seeds at $S=256$; ShapeFilter and NIST compare generation cost and success. NIST costs follow App. F.2.

Table 15: Dataset-generation cost in thousands of core-h and coverage. GNR costs are estimated from per-trajectory costs.

Task	Generation	Generation compute (10^3 core-h)			Coverage
		MPC data	GNR	Total	
HOT3D	all MPC	390.043	–	390.043	26.4%
HOT3D	all MPC data + GNR $S=256$	390.043	26.047	416.090	61.6%
HOT3D	28k MPC seeds + GNR $S=256$	116.112	32.623	148.735	59.7%
ShapeFilter	all MPC	8.541	–	8.541	93.70%
ShapeFilter	50 MPC seeds + GNR	0.203	0.562	0.766	94.82%
NIST	all MPC	79.136	–	79.136	92.19%
NIST	100 MPC seeds + GNR+MPC	25.439	19.078	44.517	92.12%

For HOT3D, the 95k MPC-success trajectories cost 390k core-h to generate, including unsuccessful attempts. Applying GNR adds 127,136 MPC-failure successes (App. D.4), giving 222,806 trajectories and 61.56% coverage of the original dataset. Using 354.1 core-s per demonstration at $S=256$ (Table 1), the additional cost is estimated as $264,806 \times 354.1/3600 = 26,047$ core-h, for a total of 416.1k core-h. This estimate uses the full sampling budget for every demonstration. The dataset grows by $2.33\times$ for 6.68% additional compute, reducing CPU cost per successful trajectory from 4.08 to 1.87 core-h ($2.18\times$).

A more practical way to deploy GNR is to use a smaller seed dataset to reduce the MPC generation cost. For generation from 28k seeds, obtaining 28,480 MPC successes requires an estimated 107,740 attempts at the measured MPC success rate of 26.43%, costing 116.1k core-h. Applying GNR to the remaining 331,670 demonstrations at 354.1 core-s each costs 32.6k core-h, for a total of 148.7k core-h. Including the seeds, this yields 215,062 trajectories (59.71% coverage), a $2.62\times$ reduction in cost relative to MPC.

For ShapeFilter, using 50 seed demonstrations reduces generation cost from 8.54k to 766 core-h while increasing dataset success from 93.7% to 94.8%. For NIST, using 100 seed demonstrations reduces generation cost from 79.14k to 44.52k core-h at about 92.2% success. The configurations and estimated costs are listed in Table 12 and Table 13.

H MULTI-EMBODIMENT EVALUATION

H.1 SETUP AND RESULTS

We train one GNR model on 147 trajectories spanning five hands and evaluate it on 100 trajectories, with 20 per hand. We use a 52-dimensional representation for actions across embodiments.

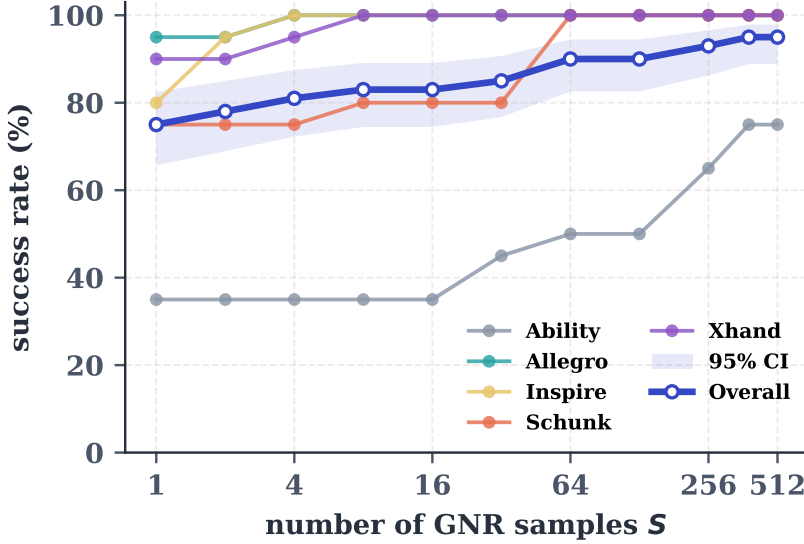


Figure 16: GNR test-time scaling across five different dexterous hands.

H.2 EFFICIENCY COMPARISON

Table 16: Multi-embodiment success on 100 test trajectories (20 per hand), with 95% Wilson confidence intervals. Samples are counted per demonstration for GNR and give the maximum budget per optimization for SPIDER.

Method	Samples	Success (% , 95% CI)
GNR	128	90.0 [82.6, 94.5]
GNR	256	93.0 [86.3, 96.6]
GNR	384	95.0 [88.8, 97.8]
SPIDER	16,384–32,768	96.0 [90.2, 98.4]

SPIDER’s total samples per episode depend on early stopping and how often MPC re-optimizes the controls during the episode.

I LEARNING FROM RETARGETED DEMONSTRATIONS

I.1 BEHAVIOR CLONING

We evaluate policy learning from retargeted trajectories (Pan et al., 2025; Chi et al., 2024; Zheng et al., 2026) using equally sized MPC, GNR+MPC, and GNR datasets with the same policy architecture and training hyperparameters.

The state-based policy is a deterministic residual multilayer perceptron with width 4096, 6 residual blocks, about 203 million parameters, and a masked regression loss. Each dataset contains 500 episodes with 108,825 frames and a mean episode length of 217.7 frames (10.9 s) at 20 Hz. We evaluate ten training checkpoints using 32 test episodes with a 400-step time limit.

The best SR achieved by the policy trained using MPC data is 37.5%, 43.75% using GNR+MPC, and 43.75% using GNR (Figure 17), demonstrating that GNR-generated data is applicable to policy learning.

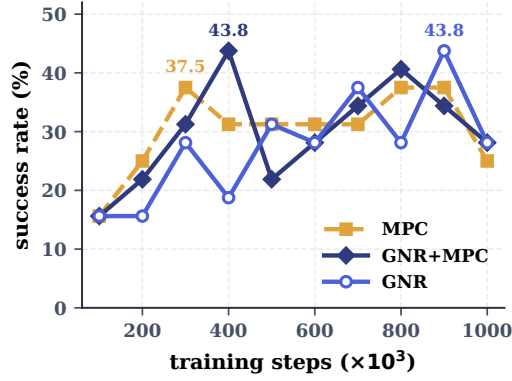


Figure 17: SR curves of behavior cloning policy training using data generated by different retargeters.

I.2 DATA FLYWHEEL

Newly retargeted demonstrations can support subsequent rounds of GNR training, forming a data flywheel (Liu et al., 2025b; Zhu et al., 2025). We test this on *ShapeFilter*, starting with 200 MPC-generated seed trajectories. Each round uses MPC to solve demonstrations on which GNR fails and adds the successful trajectories to the training set. Four rounds add 220, 104, 120, and 105 trajectories, respectively, yielding 749 training trajectories. On a fixed 256-episode test set, best-of-8 success increases from 55.2% to 74.4%, while best-of-16 success increases from 68.4% to 84.8%.

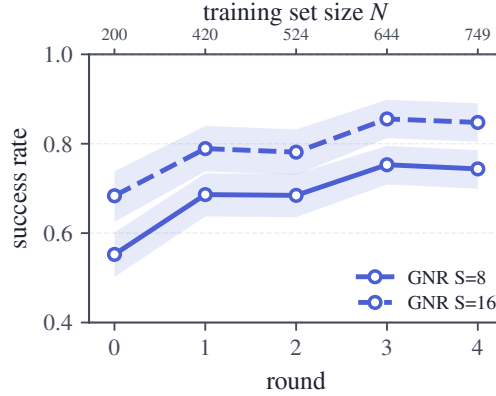


Figure 18: *ShapeFilter* success across four data-flywheel rounds.

J RETARGETING DEMOS

This section presents trajectories retargeted by different methods on ShapeFilter, NIST, and HOT3D.

J.1 SHAPEFILTER

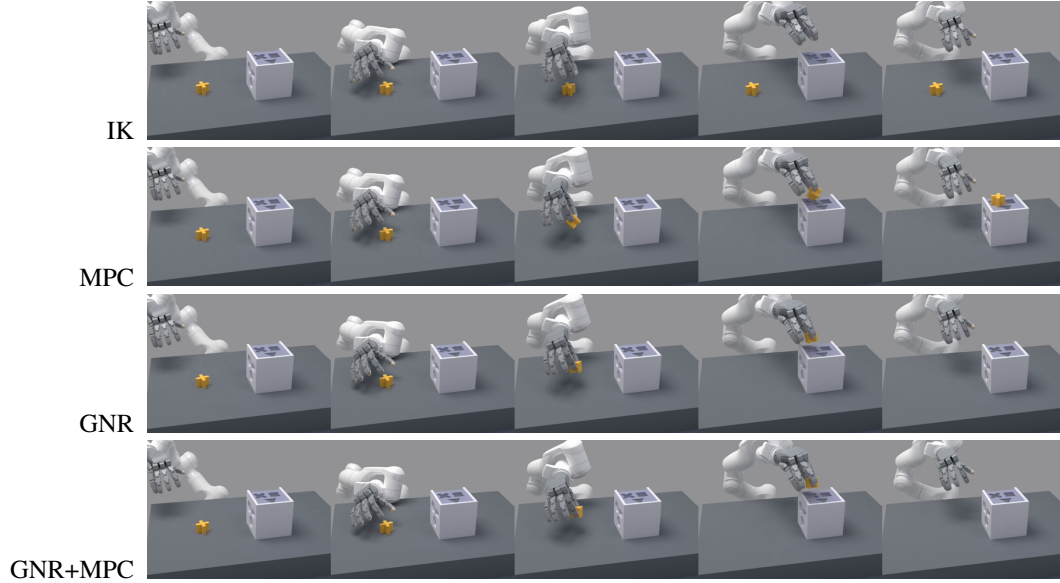


Figure 19: ShapeFilter example in which MPC fails while GNR and GNR+MPC succeed.

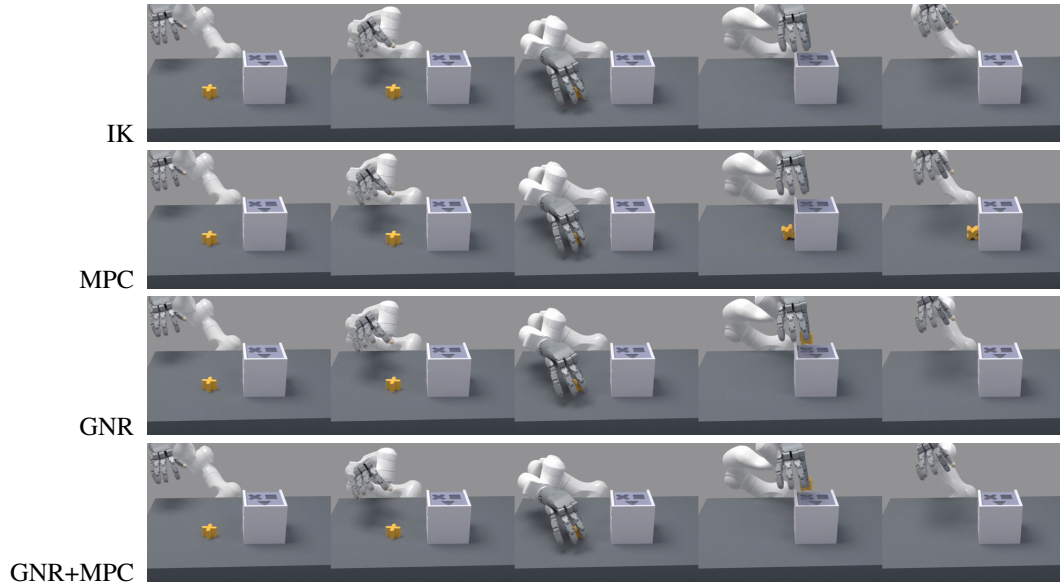


Figure 20: ShapeFilter example in which MPC fails while GNR and GNR+MPC succeed.

J.2 NIST

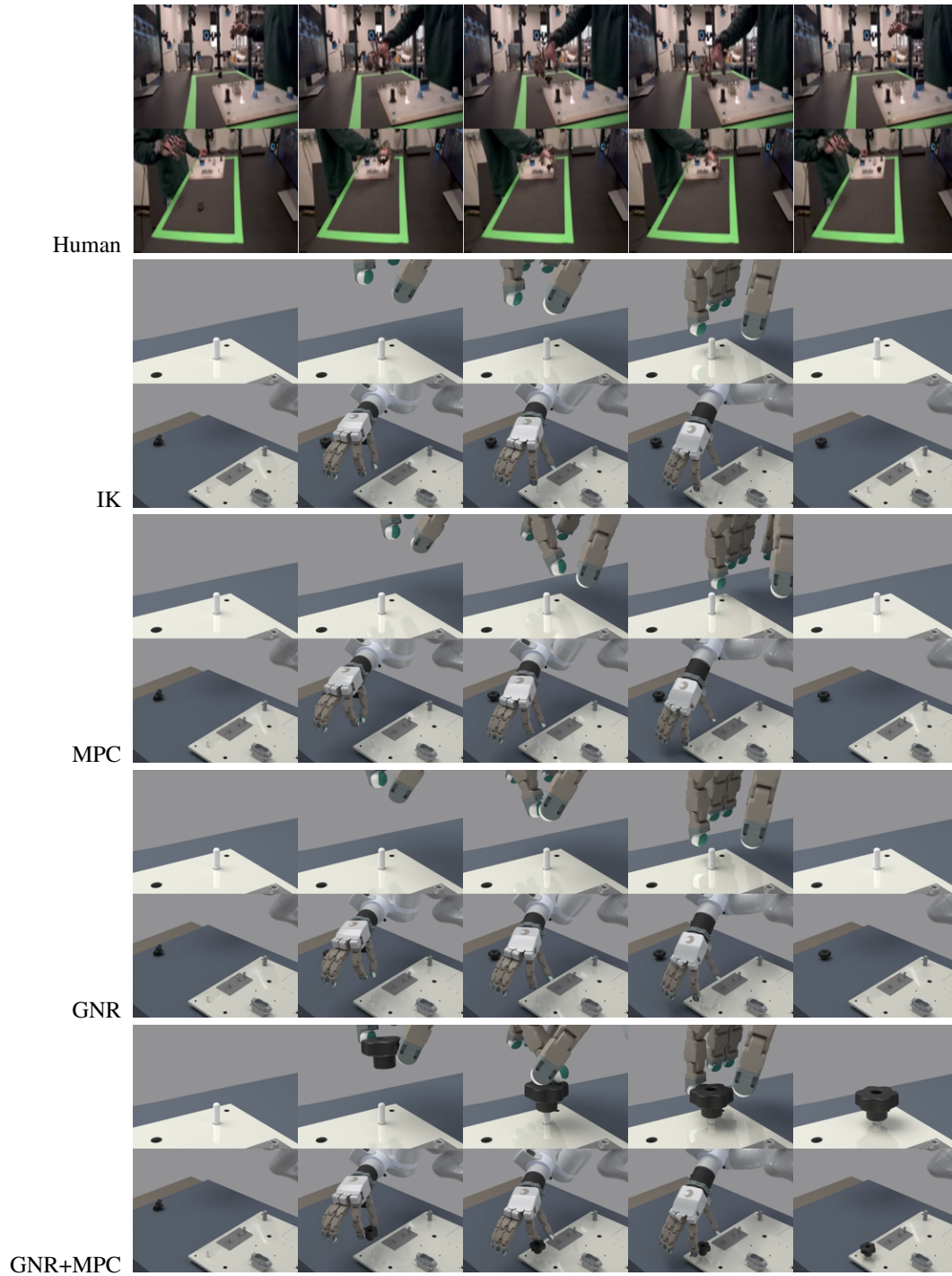


Figure 21: A NIST example. GNR+MPC succeeds where IK, MPC, and open-loop GNR fail.

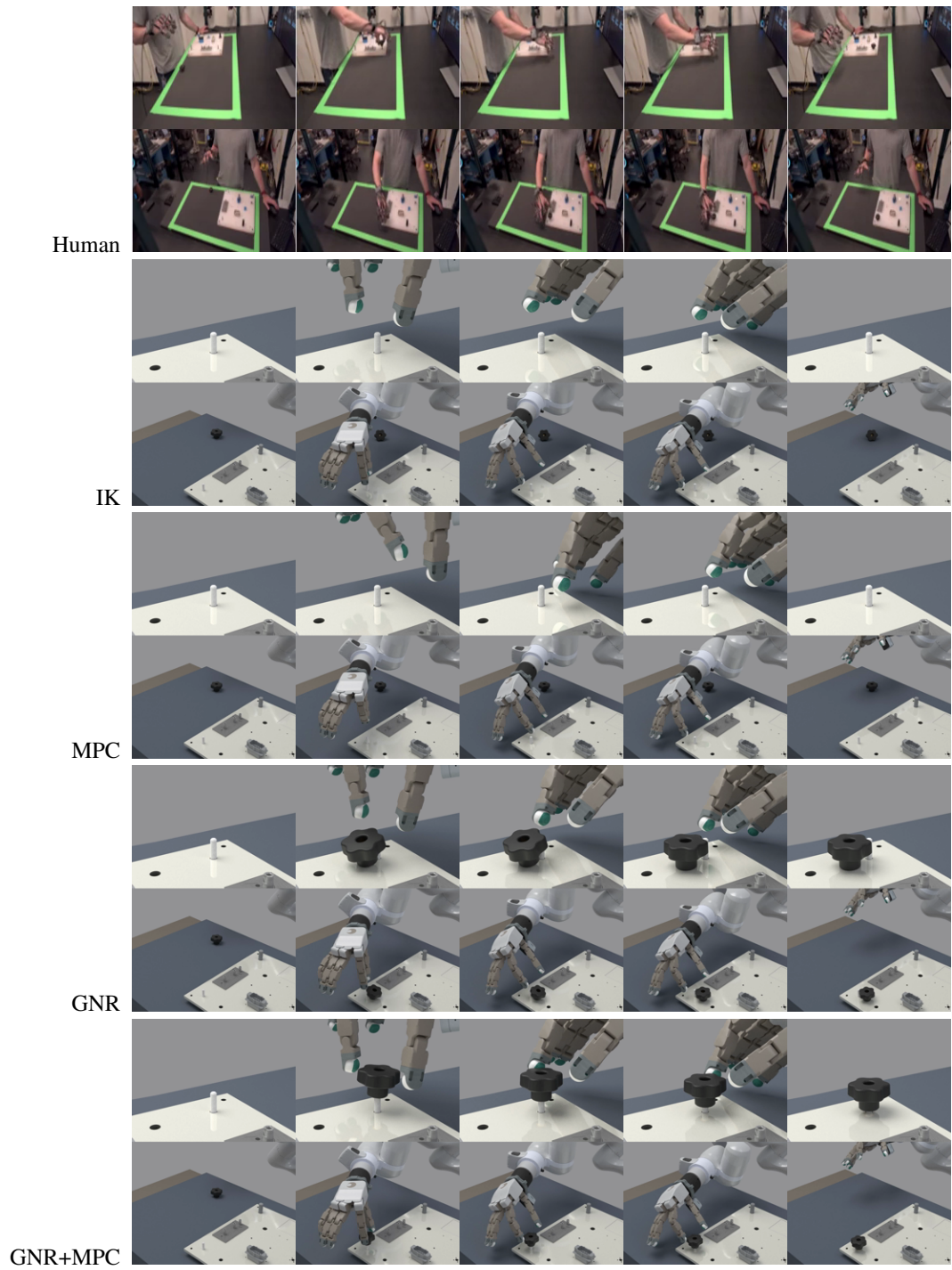


Figure 22: A NIST example. GNR+MPC succeeds where IK, MPC, and open-loop GNR fail.

J.3 HOT3D

We present egocentric recordings and retargeted trajectories across different objects.

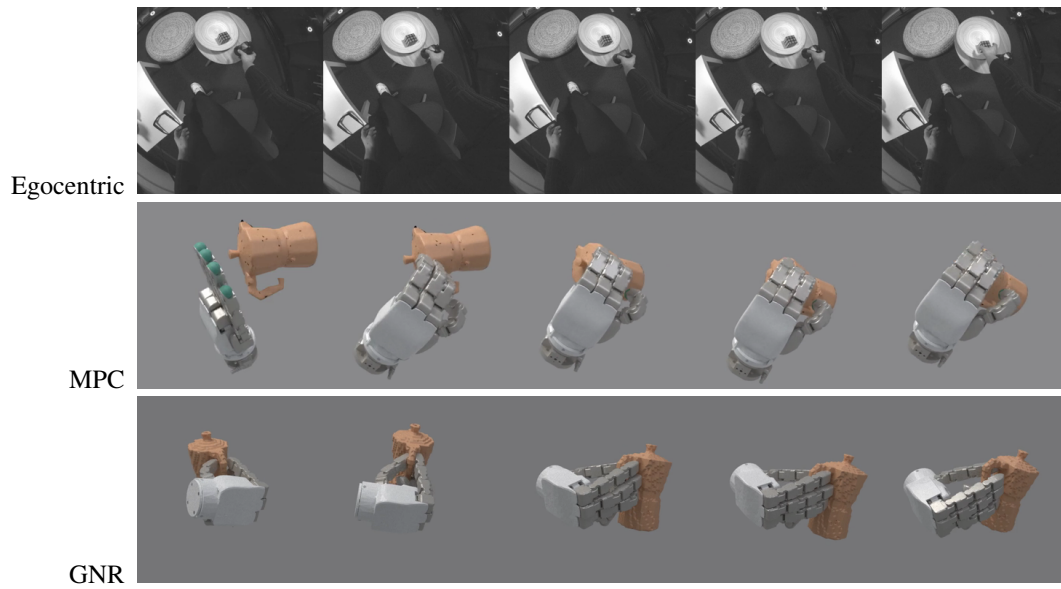


Figure 23: HOT3D coffee pot. GNR succeeds where MPC fails.

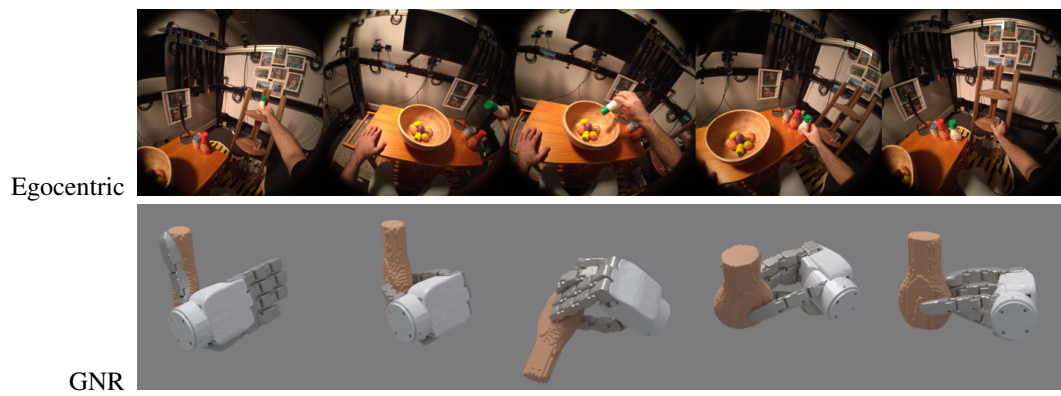


Figure 24: HOT3D ranch bottle with a stretched OOD geometry.

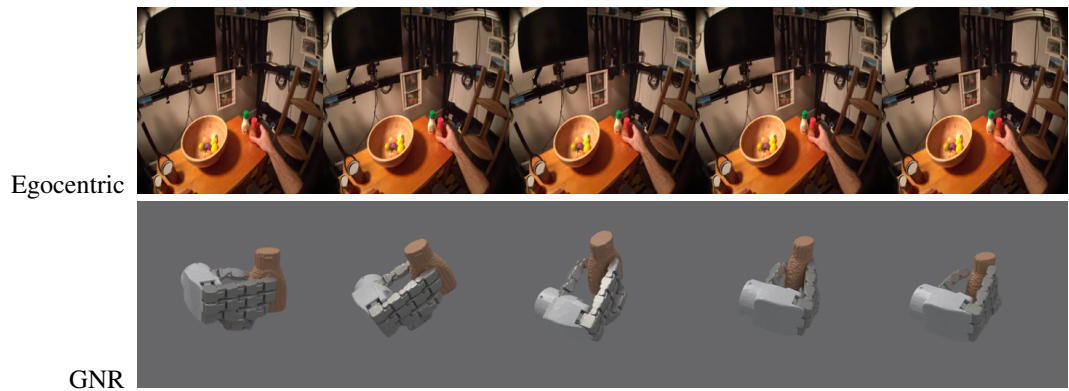


Figure 25: HOT3D barbecue sauce bottle with a stretched OOD geometry.

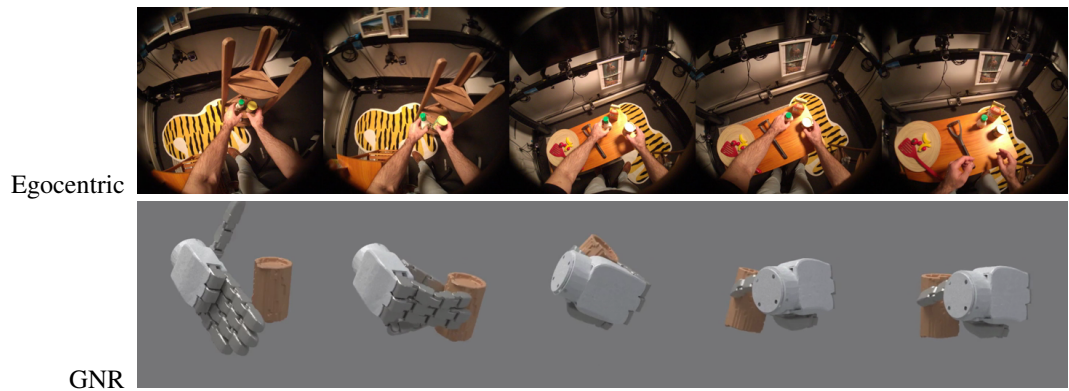


Figure 26: HOT3D parmesan can with an OOD deformation.

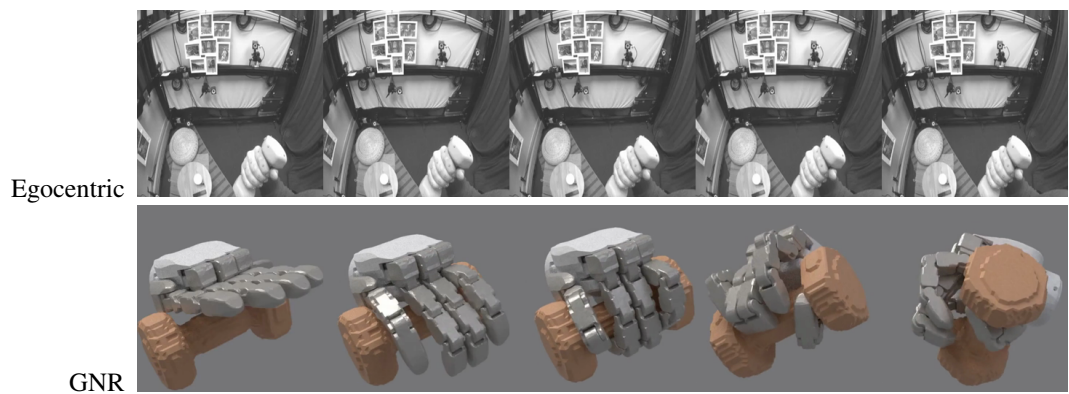


Figure 27: HOT3D dumbbell manipulation.

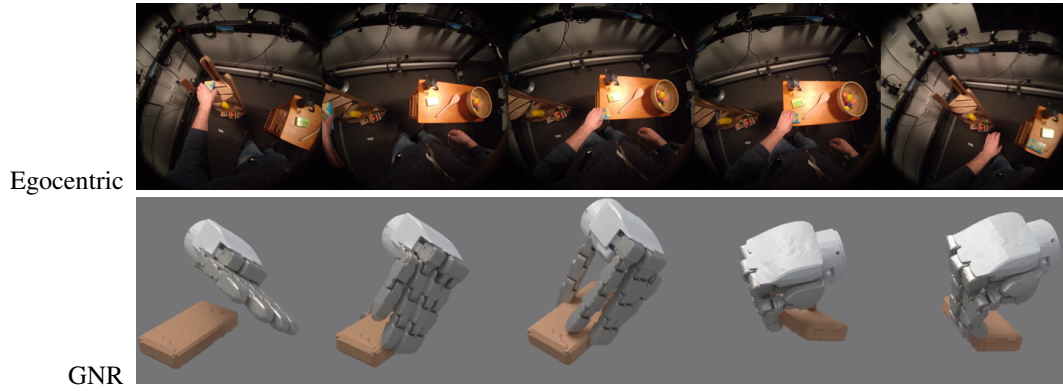


Figure 28: HOT3D waffle pick-and-place with an OOD rotation.

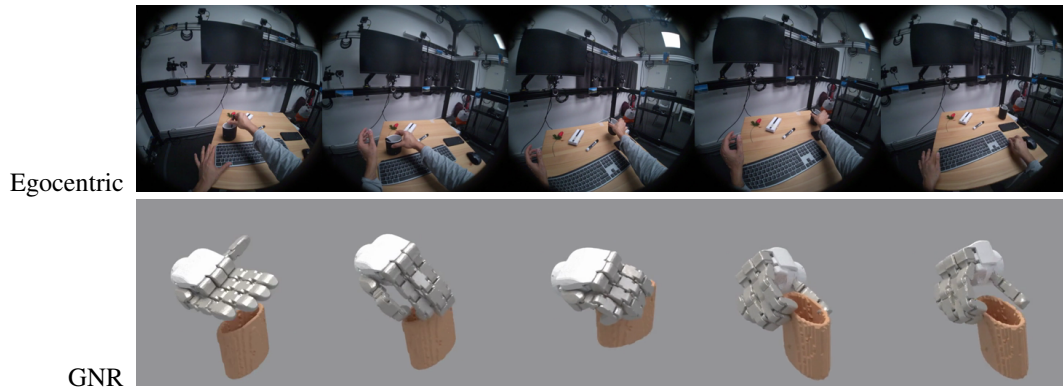


Figure 29: HOT3D holder with a stretched OOD geometry.

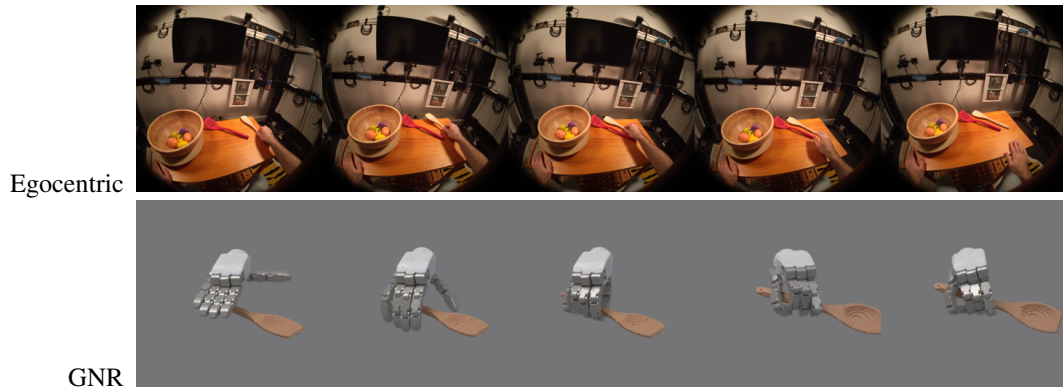


Figure 30: HOT3D wooden spoon with an OOD deformation.

J.3.1 HUMAN MOTION OPTIMIZATION

Aside from optimizing robot trajectories, we also generated 37,468 successful MPC-optimized human-motion demonstrations. The examples below show MPC making reconstructed human motion physically feasible.

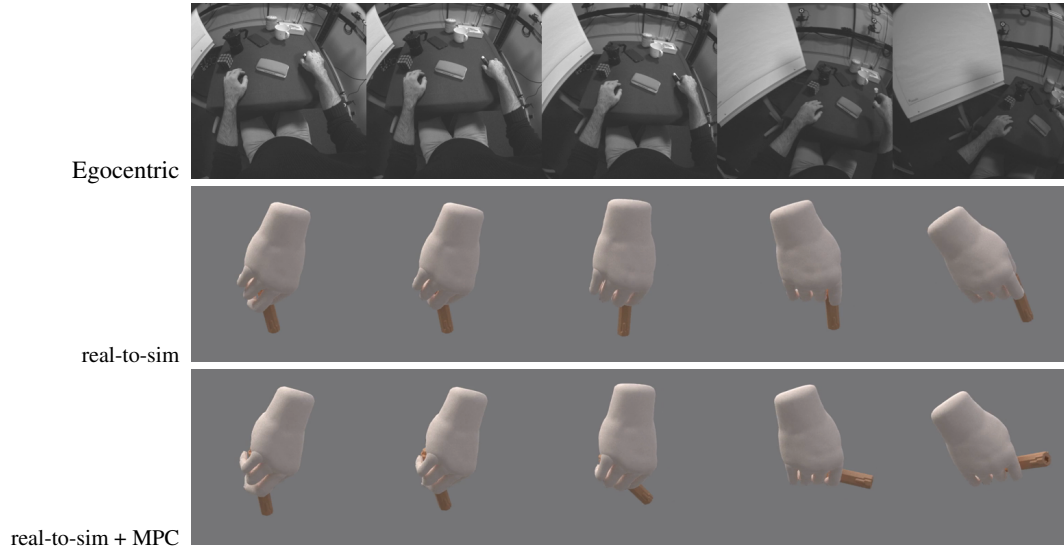


Figure 31: HOT3D whiteboard marker. MPC makes the reconstructed human motion physically feasible.

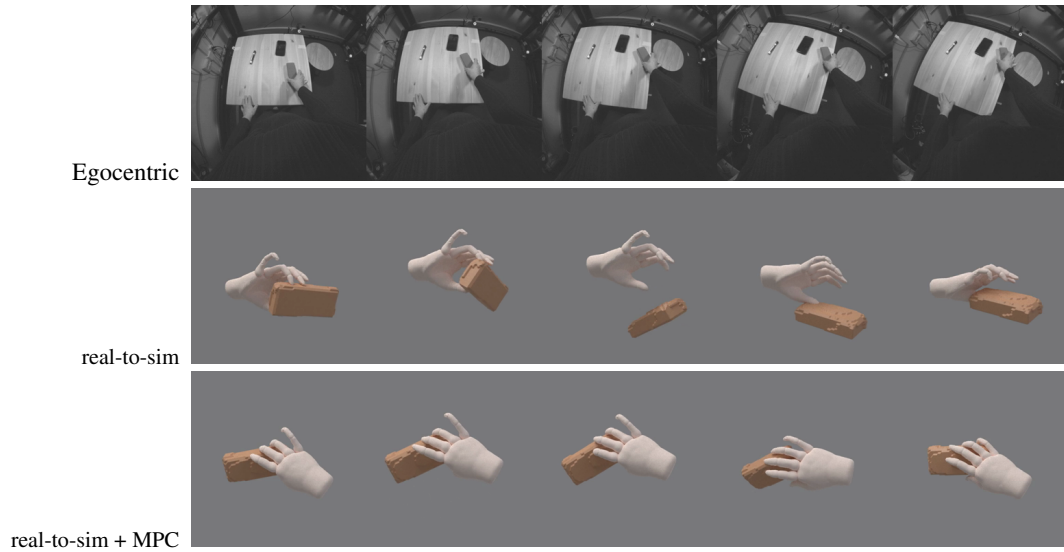


Figure 32: HOT3D whiteboard eraser. MPC makes the reconstructed human motion physically feasible.



Figure 33: HOT3D dinosaur toy. MPC makes the reconstructed human motion physically feasible.